

ELEG 4593/5593: Programming for Power Electronics

Final Project: Serial, SPRAM, PWM, ADC, and PID Buck
Converter Control

Kevin Lyon

ELEG 4593
Final Project
03 May 2026

Abstract:

This final project expands the earlier UART, SPRAM, PWM, and ADC design into a closed-loop buck converter controller. The completed system uses the LabVIEW serial interface to write and read memory-mapped registers, samples the Small PE-Eval Board feedback signals with the AD7928 ADC, and drives the buck converter gate signal DSP_G1 from a dedicated PID-controlled PWM output. The PID controller is implemented as a reusable fixed-point module and is connected to the rest of the project through a SPRAM wrapper. The wrapper reads the PWM enable, setpoint, feedback, PWM divider, and PID coefficients from SPRAM, executes one PID step at the ADC sweep rate, and converts the signed PID output into a 12-bit PWM duty command. Hardware testing showed working ADC readback and closed-loop control of the buck output.

Table of Contents:

Abstract:	2
Assignment Summary	4
System Architecture	4
PID Implementation	5
PID Wrapper Operation	6
Serial Commands For PID Coefficients	7
Hardware Results	7
Integrator Behavior	9
Conclusion	9
Appendix A: Source Code Listings	10

List of Figures:

1	LabVIEW hardware readback with the PID-controlled buck converter running. . .	8
2	LabVIEW hardware readback at a higher PID setpoint.	8

List of Tables

1	Final project memory map used by the PID and ADC system.	5
2	Default PID coefficients written at startup.	5
3	Example UART packets for reading or changing PID coefficients.	7

Assignment Summary

The final assignment required the Homework 9 serial/register/PWM/ADC interface to be expanded so that it controlled the Small PE-Eval Board's buck converter. The required top-level ports were the UART input and output, the AD7928 SPI pins, four LED/PWM outputs, and the buck gate drive output DSP_G1. The extra-credit closed-loop requirement was to incorporate a simple PI controller, include the source code in the appendix, and show hardware ADC values for the buck output.

The implemented design keeps the same general architecture used in the previous assignments. UART packets from LabVIEW are decoded by the serial controller and converted into SPRAM read/write transactions. The PWM client, ADC client, PID wrapper, and UART controller share the same SPRAM through a fixed-priority Bus_Master. The ADC client periodically stores the latest AD7928 readings beginning at address 0x0200, and the PID wrapper uses the V_Buck (0x0201) feedback register as its measured process value.

System Architecture

The top-level file, `uart_spram_top.vhd`, granted a name no longer fitting of the top module but the result of how things evolved, instantiates the internal oscillator, PLL, reset delay, UART controller, four-channel LED PWM client, ADC client, PID wrapper, Bus_Master, and generated SPRAM block. The Bus_Master uses fixed priority arbitration:

- Client 0: `pwm_spram_client`
- Client 1: `pid_spram_pwm_wrapper`
- Client 2: `adc_spram_client`
- Client 3: `uart_rs232_fifo_wrap_controller`

The LED PWM client continues to read the LED control block beginning at 0x0100. In this final version, LED1_Intensity is also used as the PID setpoint register. The ADC client writes each 12-bit ADC result into a 16-bit SPRAM word beginning at 0x0200. The PID wrapper then reads the PWM enable bit, PWM divider, coefficients, feedback, and setpoint from SPRAM before each PID calculation.

TABLE 1: Final project memory map used by the PID and ADC system.

Address	Register	Use
0x0100	PWM enable	Enables LED PWM outputs and manual integrator reset
0x0101	PWM frequency control	Controlled by LabVIEW LED_BlinkPRD
0x0103	PID setpoint	Shared with LED1_Intensity
0x0200–0x0207	ADC samples	Latest AD7928 channel results
0x0201	PID feedback	V_Buck feedback used by the PID loop
0x0300	Kp	Proportional coefficient
0x0301	Ki	Integral coefficient
0x0302	Kd	Derivative coefficient
0x0310–0x0314	PID debug	Setpoint, feedback, error, output, duty
0x0315+	Integrator debug	Full value, 16-bit words, MSB first

PID Implementation

The PID algorithm is implemented in `My_PID.vhd`. The core runs one calculation each time `execute_i` receives a rising edge. It does not continuously recompute between execute pulses. The error input is signed, while Kp, Ki, and Kd are unsigned fixed-point values. In this build, the coefficients use 10 fractional bits, so the real coefficient is

$$\text{gain} = \frac{\text{register value}}{2^{10}}. \quad (1)$$

The PID terms are calculated as

$$P = e \cdot K_p, \quad I = (\text{accumulator} \gg 8) \cdot K_i, \quad D = (e - e_{\text{previous}}) \cdot K_d. \quad (2)$$

The integrator shift of 8 is set in the top-level file when the PID wrapper is instantiated. That shift does not change the stored accumulator, but rather only reduces the accumulator's contribution to the PID output as we usually want a very small integral term and I didn't much fancy having separate gain dividers for K_p , K_i , and K_d . The following default coefficients are written into SPRAM by the PID wrapper once after reset:

TABLE 2: Default PID coefficients written at startup.

Coefficient	Address	Default Hex	Default Real Gain
Kp	0x0300	0x0050	0.078125
Ki	0x0301	0x0005	0.0048828125
Kd	0x0302	0x0000	0.0

Since K_d is zero by default, the implemented controller is operating as a PI controller unless a nonzero derivative coefficient is written through the serial interface. I put **ZERO** time into tuning this PID controller. As it is running much faster than the serial communications I would need to feed the output into an oscilloscope to tune it. It is sitting on my desk. The Scope is across the hall. It was...so far. I was grateful you didn't require us to spend that 10 minutes. However I am quite confident that my available parameter range would allow for a solidly tuned PID response.

PID Wrapper Operation

The file `pid_spram_pwm_wrapper.vhd` connects `My_PID` to the shared SPRAM bus and to a dedicated PWM generator. The wrapper first writes the default coefficients into `0x0300–0x0302`. After startup, it waits for its fixed update timer, then performs seven SPRAM reads: PWM enable, PWM frequency control, K_p , K_i , K_d , feedback, and setpoint. The update timer is hardwired from the top level to match the ADC sweep period. The setpoint comes from address `0x0103`, which corresponds to the LabVIEW `LED1_Intensity` control. The feedback comes from address `0x0201`, which is the memory-mapped `V_Buck` ADC channel used for closed-loop control.

The setpoint is a 16-bit LabVIEW value, while the ADC feedback is a 12-bit value stored in a 16-bit word. To simply map them into the same width, the wrapper uses the upper 12 bits of the setpoint:

$$setpoint_{ADC} = setpoint(15 \text{ downto } 4). \quad (3)$$

The error is then formed as

$$error = setpoint_{ADC} - feedback. \quad (4)$$

After pulsing `execute_i`, the wrapper waits for the pipelined PID output to update. Negative PID output is clipped to zero, and positive output is saturated to the 12-bit PWM range. That 12-bit duty command drives a separate `pwm_generic` instance whose output is connected directly to `DSP_G1`. The LED PWM outputs are active-low at the top-level, but `DSP_G1` is not inverted. If the PWM enable bit at `0x0100` is zero, the wrapper skips the PID execute pulse, forces the duty to zero, and clears the PID accumulator so re-enabling PWM from LabVIEW behaves like a manual integral reset.

The LabVIEW `LED_BlinkPRD` slider is reused as a bounded PWM frequency control for both the LED PWM block and the buck converter PWM. Like the other LabVIEW controls, it uses the full 16-bit range from 0 to 65535. It does not create a blink envelope. A value of 0 gives the fastest 12-bit PWM rate, approximately 6.09 kHz with the 24.93 MHz system clock. A value of 65535 maps to an effective divider of 6, or approximately 1.01 kHz. The mapping is intentionally bounded so the slider cannot accidentally force the buck converter into an unreasonable low-frequency switching mode. The PWM frequency is

$$f_{PWM} = \frac{24.93 \text{ MHz}}{\text{effective divider} \cdot 2^{12}}. \quad (5)$$

The PID executes once per ADC sweep period so it does not integrate the same stale feedback value repeatedly. The first LED output is also driven from the PID PWM signal, so LED1 mirrors the buck converter duty cycle and gives a visible indication that the PID output is changing.

Serial Commands For PID Coefficients

The UART packet format is

7E | LEN | OP_ID | COUNT | ADDR_H | ADDR_L | DATA... | CHKSUM

where 0x0A is the write command, 0x0F is the read command, and the checksum is 0xFF minus the 8-bit sum of the payload bytes. Example coefficient packets are shown in Table 3.

TABLE 3: Example UART packets for reading or changing PID coefficients.

Purpose	Packet Bytes
Read Kp, Ki, Kd	7E 04 0F 03 03 00 EA
Write Kp = 0x0050	7E 06 0A 01 03 00 00 50 A1
Write Ki = 0x0005	7E 06 0A 01 03 01 00 05 EB
Write Kd = 0x0000	7E 06 0A 01 03 02 00 00 EF
Write all defaults at once	7E 0A 0A 03 03 00 00 50 00 05 00 00 9A
Write PWM frequency control full-scale	7E 06 0A 01 01 01 FF FF F4

Hardware Results

Figure 1 shows the LabVIEW interface with the PID system running and the buck output reading approximately 1.66 V. The V_Pot readout is present and scaled correctly, confirming that the ADC path is working through the serial/SPRAM interface. The V_Buck readout is the feedback value used by the PID loop.

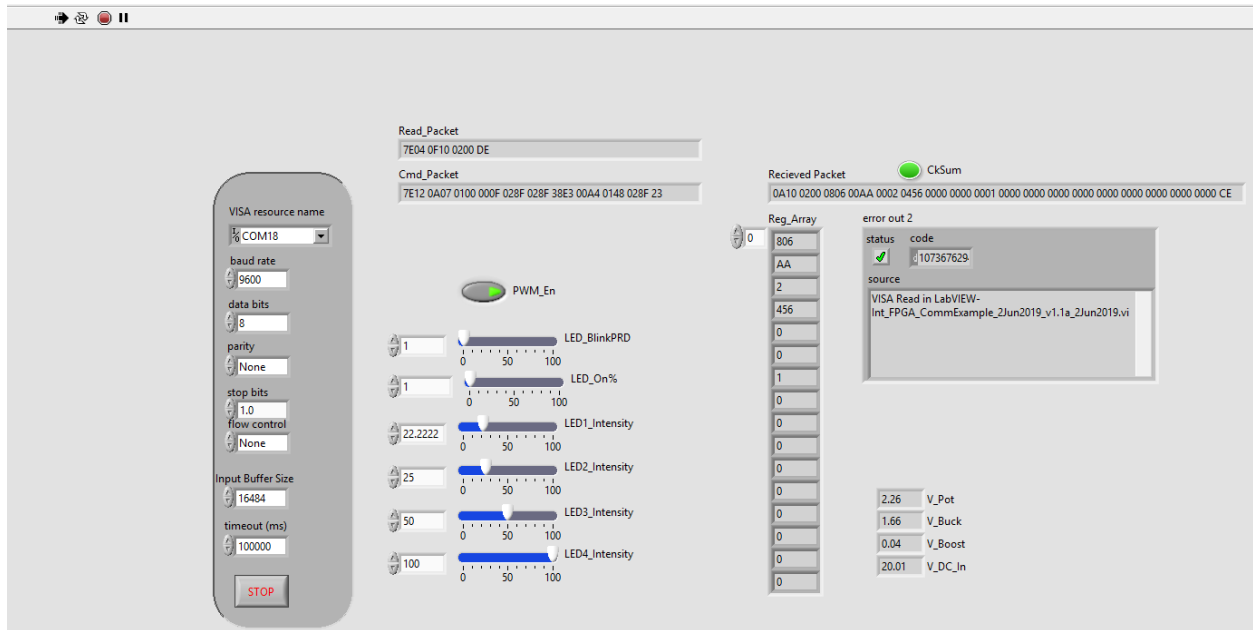
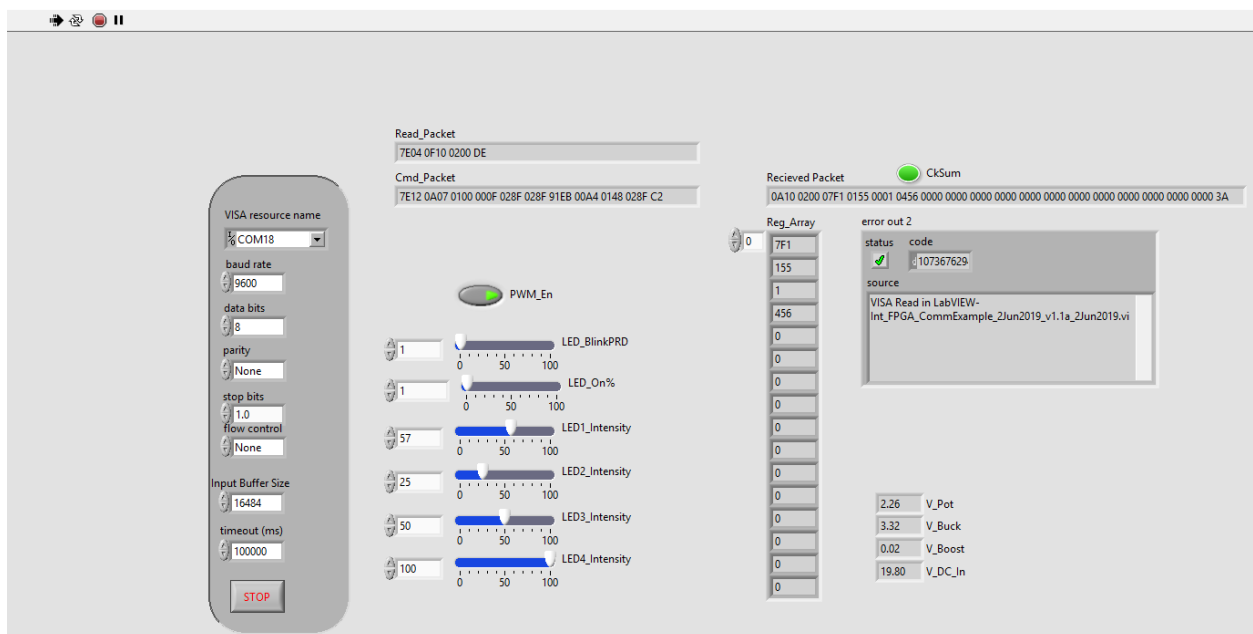


Figure 2 shows a second operating point with a higher LED1_Intensity setpoint and a measured V_Buck value of approximately 3.32 V. This demonstrates that the LabVIEW setpoint control is affecting the converter output and that the ADC values are being returned to the host interface.



Integrator Behavior

During hardware testing before the final timing change, the loop showed integrator spool-up and could slowly lose lock over time. The final code uses a 12-bit PWM period for the controlled outputs, so the fastest PWM timing reference is about

$$\frac{24.93 \text{ MHz}}{2^{12}} \approx 6.09 \text{ kHz.} \quad (6)$$

The earlier problem occurred when the PID could execute many times per ADC refresh and integrate the same stale feedback error repeatedly. The PID core does saturate its internal accumulator and output width, but the wrapper does not implement true anti-windup based on the final PWM duty saturation. Therefore, when the duty command rails high or low, the integrator may continue accumulating and then require time to unwind. This is especially noticeable if the setpoint slider is pulled all the way to full scale. In that case, the controller can demand more output than the buck converter can produce, so the integral term spools up aggressively while the PWM duty is already saturated.

A second spool condition showed up in hardware when the commanded buck voltage was set to zero. In that case the feedback remains positive while the setpoint is zero, so the error tends to stay negative. The integrator can then spool in the negative direction and make the later response feel sluggish when the setpoint is increased again, because the controller must first unwind that stored negative integral before the output rises. The final wrapper revision therefore uses the PWM enable bit at 0x0100 as a practical manual reset control: disabling PWM forces the duty cycle to zero and clears/holds the integrator, while re-enabling PWM lets the controller restart from a clean state. This is not a substitute for full anti-windup logic, but it does give the LabVIEW interface a simple and useful recovery tool during testing and was fast to implement.

Conclusion

The final project successfully combines the UART register interface, shared SPRAM bus, LED PWM client, AD7928 ADC client, and a fixed-point PID controller to drive the Small PE-Eval Board buck converter. The design meets the required serial, ADC, PWM, and DSP_G1 control functions, and the included LabVIEW screenshots show live ADC readback from the running hardware. The PID implementation is memory-mapped and tunable through the same UART packet interface used throughout the course. The final revision ties the PID calculation rate to the ADC sweep rate while leaving the former LED_BlinkPRD register available as a bounded PWM frequency control for both the LEDs and the buck converter. The first LED is driven from the same PID PWM duty as DSP_G1. Disabling the PWM enable bit now also clears and holds the integrator, which gives the LabVIEW front end a simple manual reset option when the loop has spooled in either direction. Future work should still add true anti-spooling/anti-windup logic so the integral term stops accumulating automatically when the commanded duty cycle has already reached its usable limit.

Appendix A: Source Code Listings

The following listings include the handwritten source files relevant to the final project. The generated SPRAM and PLL files are omitted because they are tool-generated. No testbench is included for this final report.

A.1 uart_spram_top.vhd

```
1  -- =====
2  -- File: uart_spram_top.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 03 March 2026
6  -- Last Updated: 03 May 2026
7  --
8  -- Description:
9  --   Top-level integration for a UART-controlled SPRAM memory interface
10 --   implemented on the Lattice MachX03D FPGA.
11 --
12 --   Earlier revisions of this design directly connected the UART protocol
13 --   parser and memory execution logic inside Bus_Master.vhd. The design
14 --   has since been refactored to separate protocol handling from the
15 --   memory arbitration layer.
16 --
17 --   The system is currently organized into the following components:
18 --
19 --       uart_rs232_fifo_wrap_controller.vhd
20 --           Handles UART RX/TX FIFOs, packet parsing, checksum validation,
21 --           command decoding, and response packet generation.
22 --
23 --       pwm_spram_client.vhd
24 --           Highest-priority memory-mapped PWM peripheral. Periodically polls
25 --           a block of SPRAM control registers and drives four PWM outputs
26 --           using locally latched shadow registers.
27 --
28 --       adc_spram_client.vhd
29 --           Middle-priority memory-mapped ADC peripheral. Periodically starts
30 --           AD7928 conversions and writes the latest valid sample into SPRAM
31 --           through the shared Bus_Master interface.
32 --
33 --       pid_spram_pwm_wrapper.vhd
34 --           PID wrapper that reads gains and process values from SPRAM,
35 --           executes My_PID at the ADC sweep rate, and drives a dedicated
36 --           runtime-divider PWM output.
37 --
38 --       Bus_Master.vhd
39 --           Dedicated prioritized SPRAM arbitration wrapper that services
40 --           read/write requests from multiple clients.
41 --
42 --       SPRAM.vhd
43 --           SCUBA-generated MachX03D on-chip SPRAM block.
44 --
45 --       pll_24m93.vhd
46 --           PLL generated from the MachX03D internal oscillator.
47 --
48 --       reset_delay.vhd
```

```

49 --           Holds global reset active for an exact number of clock cycles
50 --           after the PLL lock signal is achieved.
51 --
52 --   This separation allows the Bus_Master module to be reused in future
53 --   projects as a general-purpose memory arbitration layer while keeping
54 --   UART protocol handling isolated to the controller module. It also allows
55 --   memory-mapped peripherals, such as PWM and ADC, to source and store their
56 --   operating values directly through SPRAM without depending on any one
57 --   writer.
58 --
59 --
60 -- Top-level integration:
61 --
62 --   - MachX03D internal oscillator
63 --     clock.vhd
64 --
65 --   - PLL clock generator
66 --     pll_24m93.vhd
67 --
68 --   - Delayed reset release
69 --     reset_delay.vhd
70 --
71 --   - UART controller with RX/TX FIFOs and packet protocol
72 --     uart_rs232_fifo_wrap_controller.vhd
73 --
74 --   - Memory-mapped 4-channel PWM client
75 --     pwm_spram_client.vhd
76 --
77 --   - Memory-mapped ADC client
78 --     adc_spram_client.vhd
79 --
80 --   - Prioritized SPRAM wrapper / arbiter
81 --     Bus_Master.vhd
82 --
83 --   - On-chip SPRAM block
84 --     SPRAM.vhd
85 --
86 --
87 -- Notes:
88 --
89 --   - SPRAM is 1024 x 16 words.
90 --   - Address bus width is therefore 10 bits.
91 --   - PWM control registers begin at SPRAM address 0x0100.
92 --   - 0x0100 bit 0 is a global enable for PWM_0 through PWM_3.
93 --   - 0x0101 is used by both PWM clients as a bounded PWM frequency control.
94 --   - 0x0102 is currently unused by the LED PWM client.
95 --   - 0x0103 through 0x0106 are the PWM duty-cycle registers for PWM_0
96 --     through PWM_3 respectively.
97 --   - 0x0103 is intentionally shared with the PID setpoint.
98 --   - LED_1 / PWM_0 is driven from the PID duty output so the first LED
99 --     visibly mirrors the buck converter PWM duty.
100 --   - ADC sample registers begin at SPRAM address 0x0200 and are written to
101 --     sequential words by channel number.
102 --   - Client 0 of Bus_Master is assigned to the PWM client so PWM refresh
103 --     traffic has highest priority.
104 --   - Client 1 of Bus_Master is assigned to the PID wrapper.
105 --   - Client 2 of Bus_Master is assigned to the ADC client.
106 --   - Client 3 of Bus_Master is assigned to the UART controller.
107 --   - Global reset is held active while PLL is unlocked.

```

```

108 -- - After PLL lock is asserted, reset_delay.vhd holds reset active for
109 --   exactly RESET_DELAY_CYCLES additional clk_sys cycles.
110 -- - PWM client outputs are internally active-high and inverted at the
111 --   top-level outputs for compatibility with active-low LEDs.
112 -- - DSP_G1 is driven directly from the PID PWM output and is not inverted.
113 -- - DSP_G2 is driven low continuously.
114 -- - LED_4 and LED_5 are held inactive high to suppress faint glow.
115 -- - LED_6 and LED_7 mirror the RX and TX UART lines for activity indication.
116 --
117 --
118 -- Revision History:
119 --
120 -- 2026-03-04
121 --   - Fixed FSM timing issues caused by signal update latency when
122 --     assembling address and data words.
123 --   - Added intermediate FSM states to safely capture UART bytes
124 --     before using them in memory operations.
125 --   - Corrected write bug where the low data byte was written with
126 --     a stale value.
127 --
128 -- 2026-03-05
129 --   - Investigated RX FIFO behavior where the Empty flag did not
130 --     update as expected during command parsing.
131 --   - Replaced vendor FIFO implementation with a custom synchronous
132 --     FIFO module (My_Own_FN_FIFO.vhd).
133 --   - Updated UART FIFO wrapper to use the custom FIFO for both RX
134 --     and TX data paths.
135 --
136 -- 2026-03-16
137 --   - Major architectural refactor.
138 --   - Separated UART packet protocol logic from memory arbitration.
139 --   - Introduced uart_rs232_fifo_wrap_controller.vhd to handle
140 --     packet parsing, command execution, and response generation.
141 --   - Bus_Master.vhd redesigned as a reusable SPRAM arbitration layer.
142 --   - Top-level updated to connect UART controller -> Bus_Master -> SPRAM.
143 --   - Replaced long power-up counter reset with PLL-lock-based delayed reset.
144 --   - Added N-cycle reset release delay.
145 --
146 -- 2026-03-18
147 --   - Refactored Bus_Master client wiring to use structured request and
148 --     response records defined in bus_pkg.vhd.
149 --   - Updated the UART controller and reserved client interface signals
150 --     from flat bundles to t_bus_req / t_bus_rsp records.
151 --   - No intended functional change to top-level system behavior.
152 --
153 -- 2026-03-24
154 --   - Added pwm_spram_client.vhd as a memory-mapped PWM peripheral.
155 --   - Reserved SPRAM address 0x0004 as the PWM duty-cycle register.
156 --   - Added top-level PWM output port.
157 --   - Reassigned Bus_Master client priority so PWM is client 0 and UART
158 --     controller is client 1.
159 --   - Added periodic PWM duty refresh from SPRAM using a 10-bit PWM value.
160 --
161 -- 2026-04-05
162 --   - Removed legacy debug LED output port and associated top-level wiring.
163 --   - Removed UART controller LED debug signal routing.
164 --   - Expanded top-level PWM interface from one output to eight outputs:
165 --     PWM_0 through PWM_7.
166 --   - Updated PWM client generics to use 16-bit duty-cycle operation and

```

```

167 --      the expanded control register block beginning at 0x0100.
168 --      - No intended functional change to UART or SPRAM arbitration behavior.
169 --
170 -- 2026-04-06
171 --      - Added internal PWM raw signals.
172 --      - Inverted top-level PWM outputs for active-low LED drive.
173 --
174 -- 2026-04-07
175 --      - Reduced top-level PWM outputs from PWM_0 through PWM_7 to
176 --        PWM_0 through PWM_3 only.
177 --      - Removed PWM_4 through PWM_7 ports and associated top-level wiring.
178 --      - Added adc_spram_client.vhd to the top-level design.
179 --      - Added SPI top-level ports for the AD7928 interface.
180 --      - Reassigned Bus_Master client priority so PWM is client 0,
181 --        ADC is client 1, and UART controller is client 2.
182 --      - Added LED_4 and LED_5 outputs tied high to suppress faint glow.
183 --      - Added LED_6 and LED_7 outputs tied to RX and TX for UART activity.
184 --      - Added internal UART TX signal so TX activity can also drive LED_7
185 --        without reading back an output port.
186 --
187 -- 2026-04-22
188 --      - Added pid_spram_pwm_wrapper.vhd to the top-level design.
189 --      - Added dedicated DSP_G1 and DSP_G2 top-level outputs.
190 --      - Updated the LED PWM client so blink registers 0x0101 and 0x0102 are
191 --        ignored and 0x0100 bit 0 acts as a global enable for all four LEDs.
192 --      - Updated Bus_Master client order to:
193 --          Client 0 = PWM
194 --          Client 1 = PID
195 --          Client 2 = ADC
196 --          Client 3 = UART
197 --
198 -- 2026-05-03
199 --      - Decoupled PID calculation rate from the LabVIEW LED_BlinkPRD /
200 --        0x0101 register.
201 --      - Passed C_SAMPLE_PERIOD_CLKS into pid_spram_pwm_wrapper as
202 --        PID_UPDATE_CLKS so the PID executes once per ADC sweep period.
203 --      - Reassigned 0x0101 to control the PID PWM divider instead of the PID
204 --        update divider.
205 --      - Updated 0x0101 behavior again so it controls a bounded PWM frequency
206 --        range for both the LED PWM client and the buck/PID PWM output.
207 --      - Routed the PID PWM output to both DSP_G1 and PWM_0/LED_1 so the
208 --        first LED mirrors the buck converter duty cycle.
209 -- =====
210 library ieee;
211 use ieee.std_logic_1164.all;
212 use ieee.numeric_std.all;
213
214 library work;
215 use work.bus_pkg.all;
216
217 entity uart_spram_top is
218     generic (
219         OSC_FREQ           : string    := "8.31";      -- MHz internal oscillator setting
220         CLK_FREQ_HZ        : integer   := 24930000;    -- System clock after PLL
221         BAUD               : integer   := 9600;
222         DATA_BITS         : integer   := 8;
223         STOP_BITS          : integer   := 1;
224         PARITY              : integer   := 0;
225         RESET_DELAY_CYCLES : integer   := 16           -- extra clocks held in reset after PLL lock

```

```

226 );
227 port (
228     RX      : in  std_logic;
229     TX      : out std_logic;
230
231     SPI_Dout : in  std_logic;
232     SPI_clk  : out std_logic;
233     SPI_Din  : out std_logic;
234     SPI_CSn  : out std_logic;
235
236     PWM_0    : out std_logic;
237     PWM_1    : out std_logic;
238     PWM_2    : out std_logic;
239     PWM_3    : out std_logic;
240     DSP_G1   : out std_logic;
241     DSP_G2   : out std_logic;
242
243     LED_4    : out std_logic;
244     LED_5    : out std_logic;
245     LED_6    : out std_logic;
246     LED_7    : out std_logic
247 );
248 end entity;
249
250 architecture rtl of uart_spram_top is
251
252     -----
253     -- Clocking signals
254     -----
255     signal clk_osc      : std_logic;
256     signal clk_sys      : std_logic;
257     signal pll_lock     : std_logic;
258     signal pll_not_lock : std_logic;
259
260     -----
261     -- Global synchronous reset
262     -----
263     signal reset_i : std_logic := '1';
264
265     -----
266     -- Internal UART TX signal
267     -----
268     signal tx_uart : std_logic := '1';
269
270     -----
271     -- Bus_Master client 0 interface
272     -- Highest priority: PWM SPRAM client
273     -----
274     signal c0_req : t_bus_req := (
275         req  => '0',
276         we   => '0',
277         addr => (others => '0'),
278         wdata => (others => '0')
279     );
280
281     signal c0_rsp : t_bus_rsp := (
282         ack  => '0',
283         rdata => (others => '0')
284     );

```

```

285
286 -----
287 -- Bus_Master client 1 interface
288 -- Second priority: PID SPRAM client
289 -----
290 signal c1_req : t_bus_req := (
291     req  => '0',
292     we   => '0',
293     addr => (others => '0'),
294     wdata => (others => '0')
295 );
296
297 signal c1_rsp : t_bus_rsp := (
298     ack  => '0',
299     rdata => (others => '0')
300 );
301
302 -----
303 -- Bus_Master client 2 interface
304 -- Third priority: ADC SPRAM client
305 -----
306 signal c2_req : t_bus_req := (
307     req  => '0',
308     we   => '0',
309     addr => (others => '0'),
310     wdata => (others => '0')
311 );
312
313 signal c2_rsp : t_bus_rsp := (
314     ack  => '0',
315     rdata => (others => '0')
316 );
317
318 -----
319 -- Bus_Master client 3 interface
320 -- Lowest priority: UART controller
321 -----
322 signal c3_req : t_bus_req := (
323     req  => '0',
324     we   => '0',
325     addr => (others => '0'),
326     wdata => (others => '0')
327 );
328
329 signal c3_rsp : t_bus_rsp := (
330     ack  => '0',
331     rdata => (others => '0')
332 );
333
334 -----
335 -- SPRAM interface signals (Bus_Master / SPRAM core)
336 -----
337 signal mem_addr : std_logic_vector(9 downto 0);
338 signal mem_din  : std_logic_vector(15 downto 0);
339 signal mem_dout  : std_logic_vector(15 downto 0);
340 signal mem_we    : std_logic;
341 signal mem_en    : std_logic;
342
343 -----

```

```

344  -- Internal PWM signals before active-low output inversion
345  -----
346  signal pwm_raw : std_logic_vector(3 downto 0);
347  signal pid_pwm_raw : std_logic := '0';
348
349  -----
350  -- PWM configuration constants
351  -----
352  constant C_PWM_WIDTH      : integer := 12;
353  constant C_PWM_DIVIDER    : integer := 1;  -- ~6.1 kHz at 24.93 MHz with 12-bit PWM
354  constant C_PWM_FREQ_CTRL_MAX_DIV : integer := 6;  -- 0..65535 maps to div 1..6
355  constant C_REG_BASE_ADDR  : integer := 16#0100#;
356  constant C_PWM_PERIOD_CLKS : integer := C_PWM_DIVIDER * (2 ** C_PWM_WIDTH);
357  constant C_POLL_CYCLES    : integer := C_PWM_PERIOD_CLKS;
358  constant C_PID_PWM_WIDTH  : integer := 12;
359  constant C_PID_PWM_DIVIDER : integer := 1;  -- initial ~6.1 kHz at 24.93 MHz with 12-
        bit PWM
360  constant C_PID_INTEGRATOR_SHIFT : integer := 8;
361
362  -----
363  -- ADC configuration constants
364  -----
365  --Channel 0 is ADC_Vin
366  --Channel 1 is ADC_VBuck
367  --Channel 2 is ADC_VBoost
368  --Channel 3 is ADC_VPot
369  constant C_ADC_CLK_DIV      : integer := 5;
370  constant C_SAMPLE_PERIOD_CLKS : integer := C_PWM_PERIOD_CLKS;
371  constant C_ADC_BASE_ADDR    : integer := 16#0200#;
372
373  begin
374
375  pll_not_lock <= not pll_lock;
376
377  -----
378  -- UART top-level output
379  -----
380  TX <= tx_uart;
381
382  -----
383  -- Active-low LED output inversion
384  -----
385  PWM_0 <= not pid_pwm_raw;
386  PWM_1 <= not pwm_raw(1);
387  PWM_2 <= not pwm_raw(2);
388  PWM_3 <= not pwm_raw(3);
389  DSP_G1 <= pid_pwm_raw;
390  DSP_G2 <= '0';
391
392  -----
393  -- Additional LED assignments
394  -----
395  LED_4 <= '1';
396  LED_5 <= '1';
397  LED_6 <= RX;
398  LED_7 <= tx_uart;
399
400  -----
401  -- Internal oscillator clock module

```

```

402 -----
403 u_clk : entity work.clock
404     generic map (
405         OSC_FREQ => OSC_FREQ
406     )
407     port map (
408         clk => clk_osc
409     );
410
411 -----
412 -- PLL:
413 -- Convert internal oscillator clock to the desired system clock
414 -----
415 u_pll : entity work pll_24m93
416     port map (
417         CLKI  => clk_osc,
418         CLKOP => clk_sys,
419         LOCK  => pll_lock
420     );
421
422 -----
423 -- Reset delay
424 -----
425 u_reset_delay : entity work.reset_delay
426     generic map (
427         DELAY_CYCLES => RESET_DELAY_CYCLES
428     )
429     port map (
430         clk      => clk_sys,
431         rst_in   => pll_not_lock,
432         rst_out  => reset_i
433     );
434
435 -----
436 -- PWM SPRAM client
437 -- Highest-priority Bus_Master client
438 -----
439 u_pwm_client : entity work.pwm_spram_client
440     generic map (
441         CLK_FREQ_HZ    => CLK_FREQ_HZ,
442         PWM_WIDTH      => C_PWM_WIDTH,
443         PWM_DIVIDER    => C_PWM_DIVIDER,
444         PWM_FREQ_CTRL_MAX_DIV => C_PWM_FREQ_CTRL_MAX_DIV,
445         REG_BASE_ADDR  => C_REG_BASE_ADDR,
446         POLL_CYCLES    => C_POLL_CYCLES
447     )
448     port map (
449         clk_sys => clk_sys,
450         reset_i => reset_i,
451
452         bus_req => c0_req,
453         bus_rsp => c0_rsp,
454
455         PWM_0 => pwm_raw(0),
456         PWM_1 => pwm_raw(1),
457         PWM_2 => pwm_raw(2),
458         PWM_3 => pwm_raw(3)
459     );
460

```

```

461 -----
462 -- PID SPRAM + PWM wrapper
463 -- Second-priority Bus_Master client
464 -----
465 u_pid_client : entity work.pid_spram_pwm_wrapper
466   generic map (
467     PWM_COUNTER_WIDTH => C_PID_PWM_WIDTH,
468     PWM_DIVIDER       => C_PID_PWM_DIVIDER,
469     PWM_FREQ_CTRL_MAX_DIV => C_PWM_FREQ_CTRL_MAX_DIV,
470     PID_UPDATE_CLKS    => C_SAMPLE_PERIOD_CLKS,
471     PID_INTEGRATOR_SHIFT => C_PID_INTEGRATOR_SHIFT
472   )
473   port map (
474     clk_sys  => clk_sys,
475     reset_i  => reset_i,
476     bus_req  => c1_req,
477     bus_rsp  => c1_rsp,
478     pid_pwm_o => pid_pwm_raw
479   );
480
481 -----
482 -- ADC SPRAM client
483 -- Third-priority Bus_Master client
484 -----
485 u_adc_client : entity work.adc_spram_client
486   generic map (
487     CLK_FREQ_HZ      => CLK_FREQ_HZ,
488     ADC_CLK_DIV      => C_ADC_CLK_DIV,
489     SAMPLE_PERIOD_CLKS => C_SAMPLE_PERIOD_CLKS,
490     ADC_BASE_ADDR    => C_ADC_BASE_ADDR
491   )
492   port map (
493     clk_sys => clk_sys,
494     reset_i => reset_i,
495
496     bus_req => c2_req,
497     bus_rsp => c2_rsp,
498
499     adc_cs_n_o => SPI_CS_n,
500     adc_sclk_o => SPI_clk,
501     adc_din_o  => SPI_Din,
502     adc_dout_i => SPI_Dout
503   );
504
505 -----
506 -- UART protocol controller
507 -- Lowest-priority Bus_Master client
508 -----
509 u_uart_ctrl : entity work. uart_rs232_fifo_wrap_controller
510   generic map (
511     CLK_FREQ_HZ => CLK_FREQ_HZ,
512     BAUD        => BAUD,
513     DATA_BITS  => DATA_BITS,
514     STOP_BITS   => STOP_BITS,
515     PARITY      => PARITY
516   )
517   port map (
518     clk_sys => clk_sys,
519     reset_i => reset_i,

```

```

520
521     RX => RX,
522     TX => tx_uart,
523
524     bus_req => c3_req,
525     bus_rsp => c3_rsp
526 );
527
528 -----
529 -- Dedicated SPRAM wrapper / arbiter
530 -- Priority: client 0 > client 1 > client 2 > client 3
531 -----
532 u_bus : entity work.Bus_Master
533 port map (
534     clk_sys => clk_sys,
535     reset_i => reset_i,
536
537     -- Client 0: PWM client
538     c0_req => c0_req,
539     c0_rsp => c0_rsp,
540
541     -- Client 1: PID wrapper
542     c1_req => c1_req,
543     c1_rsp => c1_rsp,
544
545     -- Client 2: ADC client
546     c2_req => c2_req,
547     c2_rsp => c2_rsp,
548
549     -- Client 3: UART controller
550     c3_req => c3_req,
551     c3_rsp => c3_rsp,
552
553     -- SPRAM side
554     mem_addr => mem_addr,
555     mem_din  => mem_din,
556     mem_dout => mem_dout,
557     mem_we   => mem_we,
558     mem_en   => mem_en
559 );
560
561 -----
562 -- SPRAM core (SCUBA generated)
563 -----
564 u_spram : entity work.SPRAM
565 port map (
566     Address => mem_addr,
567     Data    => mem_din,
568     Clock   => clk_sys,
569     WE      => mem_we,
570     ClockEn => mem_en,
571     Q       => mem_dout
572 );
573
574 end architecture;

```

A.2 pid_spram_pwm_wrapper.vhd

```
1  -- =====
2  -- File: pid_spram_pwm_wrapper.vhd
3  --
4  -- Creator: Kevin Lyon / Codex
5  -- Date Created: 22 April 2026
6  -- Last Updated: 03 May 2026
7  --
8  -- Description:
9  --   Memory-mapped PID + PWM wrapper for the shared SPRAM / Bus_Master
10 --   architecture.
11 --
12 --   This module reads PID configuration and live process values from SPRAM,
13 --   executes one step of My_PID at a fixed hardware update rate matched to the
14 --   ADC sweep period, and converts the resulting signed PID output into the
15 --   duty-cycle input of a dedicated runtime-divider PWM generator.
16 --
17 -- Register map used by this wrapper:
18 --
19 --   0x0100 : PWM enable
20 --           Bit 0 is treated as the overall PWM enable. When it is zero,
21 --           this wrapper forces the buck PWM duty low and clears/holds the
22 --           PID integrator so toggling the LabVIEW enable acts as a manual
23 --           integral reset.
24 --   0x0101 : PID PWM frequency control
25 --           0 => fastest PWM
26 --           65535 => slowest PWM in the bounded configured range
27 --
28 --           PWM frequency is:
29 --           f_pwm = clk_sys / (effective_divider * 2^PWM_COUNTER_WIDTH)
30 --
31 --   0x0103 : PID setpoint
32 --           Shared intentionally with the PWM_0 duty register
33 --   0x0201 : PID feedback input
34 --   0x0300 : Kp
35 --   0x0301 : Ki
36 --   0x0302 : Kd
37 --   0x0310 : Debug scaled setpoint (12-bit ADC-domain value, zero-extended)
38 --   0x0311 : Debug feedback
39 --   0x0312 : Debug error
40 --   0x0313 : Debug PID output (low 16 bits after saturation to 16-bit signed)
41 --   0x0314 : Debug PWM duty (zero-extended)
42 --   0x0315+ : Debug PID accumulator image in 16-bit words, most-significant
43 --             word first. The number of words depends on PID_ACCUM_WIDTH.
44 --
45 -- Fixed-point interpretation of Kp / Ki / Kd:
46 --
47 --   The coefficient registers are treated as unsigned fixed-point values with
48 --   PID_COEFF_FRAC_BITS fractional bits:
49 --
50 --           real_gain = register_value / 2^PID_COEFF_FRAC_BITS
51 --
52 -- Default example with PID_COEFF_FRAC_BITS = 10:
53 --
54 --           0x0400 = 1.0
55 --           0x0200 = 0.5
56 --           0x0100 = 0.25
57 --
```

```

58 -- Startup defaults written by this wrapper into SPRAM once after reset:
59 --
60 --     Kp = 0x0050
61 --     Ki = 0x0005
62 --     Kd = 0x0000 = 0.0
63 --
64 -- Integral-term scaling:
65 --
66 --     The instantiated My_PID core also applies INTEGRATOR_SHIFT, which shifts
67 --     the stored accumulator right before the I-term multiply. Larger values
68 --     make the integral contribution build more gently.
69 --
70 -- Notes:
71 -- * error = scaled_setpoint - feedback
72 -- * The 16-bit setpoint register is scaled to the ADC domain by dividing by
73 --   16 before subtraction, so:
74 --     0x0000 => target 0
75 --     0xFFFF => target 4095 (full-scale 12-bit ADC)
76 -- * The feedback and setpoint words are interpreted as unsigned 16-bit
77 --   magnitudes and expanded to signed math for subtraction.
78 -- * Negative PID output is clipped to zero before driving PWM duty.
79 -- * Positive PID output is saturated to the available PWM range.
80 -- * On startup, this wrapper writes conservative default PID gains into
81 --   SPRAM before beginning normal PID updates.
82 -- * The wrapper re-reads the PWM enable, PWM divider, Kp, Ki, Kd, feedback, and
83 --   setpoint before each PID update.
84 -- * If the live Ki register is read as zero, this wrapper pulses the PID
85 --   core's accumulator-clear input before executing the next PID step.
86 -- * If the live PWM enable bit at 0x0100 is zero, this wrapper skips the
87 --   PID execute pulse, forces duty to zero, and repeatedly clears the
88 --   integrator so disabling PWM from LabVIEW can be used as a manual reset.
89 -- * The update scheduler is free-running and intentionally decoupled from
90 --   the PWM frequency. It is hardwired by PID_UPDATE_CLKS so the PID can be
91 --   matched to the ADC sweep/update rate.
92 -- * Register 0x0101 was changed on 03 May 2026 from a PID update divider
93 --   into a bounded 16-bit runtime PWM frequency control so the LabVIEW
94 --   LED_BlinkPRD control can adjust buck switching frequency without
95 --   changing PID sample rate.
96 -- * My_PID is internally pipelined, so this wrapper waits the required
97 --   extra clocks before latching the updated PID output into the PWM duty
98 --   register. One additional wrapper wait state is included so the PID
99 --   output register has completed its clocked update before duty_i is
100 --   sampled here.
101 -- =====
102
103 library ieee;
104 use ieee.std_logic_1164.all;
105 use ieee.numeric_std.all;
106
107 library work;
108 use work.bus_pkg.all;
109
110 entity pid_spram_pwm_wrapper is
111     generic (
112         PID_ERROR_WIDTH      : positive := 17;
113         PID_COEFF_WIDTH      : positive := 16;
114         PID_COEFF_FRAC_BITS  : natural  := 10;
115         PID_INTEGRATOR_SHIFT : natural  := 0;
116         PID_ACCUM_WIDTH      : positive := 32;

```

```

117     PID_OUTPUT_WIDTH      : positive := 24;
118     PWM_COUNTER_WIDTH     : positive := 12;
119     PWM_DIVIDER           : positive := 1;
120     PWM_FREQ_CTRL_MAX_DIV : positive := 6;
121     PID_UPDATE_CLKS       : positive := 65536;
122     PWM_DIV_ADDR          : integer  := 16#0101#;
123     SETPOINT_ADDR        : integer  := 16#0103#;
124     FEEDBACK_ADDR        : integer  := 16#0201#;
125     KP_ADDR               : integer  := 16#0300#;
126     KI_ADDR               : integer  := 16#0301#;
127     KD_ADDR               : integer  := 16#0302#
128 );
129 port (
130     clk_sys    : in  std_logic;
131     reset_i    : in  std_logic;
132     bus_req    : out t_bus_req;
133     bus_rsp    : in  t_bus_rsp;
134     pid_pwm_o  : out std_logic
135 );
136 end entity;
137
138 architecture rtl of pid_spram_pwm_wrapper is
139
140     constant C_NUM_READS      : integer := 7;
141     constant C_DBG_FIXED_WORDS : integer := 5;
142     constant C_DBG_ACCUM_WORDS : integer := (PID_ACCUM_WIDTH + 15) / 16;
143     constant C_DBG_WORDS_TOTAL : integer := C_DBG_FIXED_WORDS + C_DBG_ACCUM_WORDS;
144     constant C_KP_INIT        : std_logic_vector(15 downto 0) := x"0050";
145     constant C_KI_INIT        : std_logic_vector(15 downto 0) := x"0005";
146     constant C_KD_INIT        : std_logic_vector(15 downto 0) := x"0000";
147
148     type t_state is (
149         ST_INIT_WRITE,
150         ST_INIT_WAIT_ACK,
151         ST_INIT_GAP,
152         ST_IDLE,
153         ST_REQ_READ,
154         ST_WAIT_ACK,
155         ST_READ_CAPTURE,
156         ST_REQ_GAP,
157         ST_HOLD_DISABLED,
158         ST_CLEAR_ACCUM,
159         ST_EXEC_PULSE,
160         ST_WAIT_PID_1,
161         ST_WAIT_PID_2,
162         ST_WAIT_PID_3,
163         ST_LATCH_PWM,
164         ST_DBG_WRITE,
165         ST_DBG_WAIT_ACK,
166         ST_DBG_GAP
167     );
168
169     signal state : t_state := ST_IDLE;
170
171     signal bus_req_r : t_bus_req := (
172         req  => '0',
173         we   => '0',
174         addr => (others => '0'),
175         wdata => (others => '0')

```

```

176 );
177
178 signal read_index_r      : integer range 0 to C_NUM_READS-1 := 0;
179 signal read_data_r      : std_logic_vector(15 downto 0) := (others => '0');
180 signal update_period_count_r : integer range 0 to PID_UPDATE_CLKS-1 := 0;
181 signal update_tick_r     : std_logic := '0';
182
183 signal update_pending_r : std_logic := '0';
184 signal pwm_enable_r     : std_logic := '0';
185 signal pwm_divider_r    : unsigned(15 downto 0) := to_unsigned(PWM_DIVIDER, 16);
186
187 signal setpoint_r : std_logic_vector(15 downto 0) := (others => '0');
188 signal feedback_r : std_logic_vector(15 downto 0) := (others => '0');
189 signal kp_r       : unsigned(PID_COEFF_WIDTH-1 downto 0) := (others => '0');
190 signal ki_r       : unsigned(PID_COEFF_WIDTH-1 downto 0) := (others => '0');
191 signal kd_r       : unsigned(PID_COEFF_WIDTH-1 downto 0) := (others => '0');
192 signal setpoint_adc_s : unsigned(11 downto 0);
193
194 signal pid_error_s      : signed(PID_ERROR_WIDTH-1 downto 0);
195 signal pid_execute_r    : std_logic := '0';
196 signal pid_clear_accum_r : std_logic := '0';
197 signal pid_output_s     : signed(PID_OUTPUT_WIDTH-1 downto 0);
198 signal pwm_duty_r       : unsigned(PWM_COUNTER_WIDTH-1 downto 0) := (others => '0');
199 signal pwm_count_r      : unsigned(PWM_COUNTER_WIDTH-1 downto 0) := (others => '0');
200 signal pwm_duty_latched_r : unsigned(PWM_COUNTER_WIDTH-1 downto 0) := (others => '0');
201 signal pwm_div_count_r  : unsigned(15 downto 0) := (others => '0');
202 signal pid_pwm_r        : std_logic := '0';
203 signal init_index_r     : integer range 0 to 2 := 0;
204 signal dbg_index_r      : integer range 0 to C_DBG_WORDS_TOTAL-1 := 0;
205 signal dbg_setpoint_r   : std_logic_vector(15 downto 0) := (others => '0');
206 signal dbg_feedback_r   : std_logic_vector(15 downto 0) := (others => '0');
207 signal dbg_error_r      : std_logic_vector(15 downto 0) := (others => '0');
208 signal dbg_pid_r        : std_logic_vector(15 downto 0) := (others => '0');
209 signal dbg_pwm_r        : std_logic_vector(15 downto 0) := (others => '0');
210 signal pid_accum_s      : signed(PID_ACCUM_WIDTH-1 downto 0);
211
212 function f_read_addr(
213     idx : integer
214 ) return std_logic_vector is
215 begin
216     case idx is
217         when 0 =>
218             return std_logic_vector(to_unsigned(16#0100#, 10));
219         when 1 =>
220             return std_logic_vector(to_unsigned(PWM_DIV_ADDR, 10));
221         when 2 =>
222             return std_logic_vector(to_unsigned(KP_ADDR, 10));
223         when 3 =>
224             return std_logic_vector(to_unsigned(KI_ADDR, 10));
225         when 4 =>
226             return std_logic_vector(to_unsigned(KD_ADDR, 10));
227         when 5 =>
228             return std_logic_vector(to_unsigned(FEEDBACK_ADDR, 10));
229         when others =>
230             return std_logic_vector(to_unsigned(SETPOINT_ADDR, 10));
231     end case;
232 end function;
233
234 function f_freq_ctrl_to_div(

```

```

235     value_in : std_logic_vector(15 downto 0)
236 ) return unsigned is
237     variable ctrl_v : integer;
238     variable div_v  : integer;
239 begin
240     ctrl_v := to_integer(unsigned(value_in));
241
242     if PWM_FREQ_CTRL_MAX_DIV <= 1 then
243         div_v := 1;
244     else
245         div_v := 1 + ((ctrl_v * (PWM_FREQ_CTRL_MAX_DIV - 1) + 32767) / 65535);
246     end if;
247
248     return to_unsigned(div_v, 16);
249 end function;
250
251 function f_sat_to_pwm(
252     value_in      : signed;
253     target_size   : positive
254 ) return unsigned is
255     variable result_v : unsigned(target_size-1 downto 0);
256     variable fits_v   : boolean;
257 begin
258     if value_in(value_in'high) = '1' then
259         result_v := (others => '0');
260     elsif value_in'length <= target_size then
261         result_v := resize(unsigned(value_in), target_size);
262     else
263         fits_v := true;
264
265         for idx in value_in'high downto target_size loop
266             if value_in(idx) = '1' then
267                 fits_v := false;
268             end if;
269         end loop;
270
271         if fits_v then
272             result_v := unsigned(value_in(target_size-1 downto 0));
273         else
274             result_v := (others => '1');
275         end if;
276     end if;
277
278     return result_v;
279 end function;
280
281 function f_init_addr(
282     idx : integer
283 ) return std_logic_vector is
284 begin
285     case idx is
286         when 0 =>
287             return std_logic_vector(to_unsigned(KP_ADDR, 10));
288         when 1 =>
289             return std_logic_vector(to_unsigned(KI_ADDR, 10));
290         when others =>
291             return std_logic_vector(to_unsigned(KD_ADDR, 10));
292     end case;
293 end function;

```

```

294
295 function f_init_data(
296     idx : integer
297 ) return std_logic_vector is
298 begin
299     case idx is
300         when 0 =>
301             return C_KP_INIT;
302         when 1 =>
303             return C_KI_INIT;
304         when others =>
305             return C_KD_INIT;
306     end case;
307 end function;
308
309 function f_dbg_addr(
310     idx : integer
311 ) return std_logic_vector is
312 begin
313     case idx is
314         when 0 =>
315             return std_logic_vector(to_unsigned(16#0310#, 10));
316         when 1 =>
317             return std_logic_vector(to_unsigned(16#0311#, 10));
318         when 2 =>
319             return std_logic_vector(to_unsigned(16#0312#, 10));
320         when 3 =>
321             return std_logic_vector(to_unsigned(16#0313#, 10));
322         when 4 =>
323             return std_logic_vector(to_unsigned(16#0314#, 10));
324         when others =>
325             return std_logic_vector(to_unsigned(16#0315# + (idx - C_DBG_FIXED_WORDS), 10));
326     end case;
327 end function;
328
329 function f_dbg_data(
330     idx          : integer;
331     setpoint     : std_logic_vector(15 downto 0);
332     feedback     : std_logic_vector(15 downto 0);
333     err_word     : std_logic_vector(15 downto 0);
334     pid_word     : std_logic_vector(15 downto 0);
335     pwm_word     : std_logic_vector(15 downto 0);
336     accum_word   : signed
337 ) return std_logic_vector is
338     variable word_idx_v : integer;
339     variable accum_ext_v : signed(C_DBG_ACCUM_WORDS*16-1 downto 0);
340 begin
341     case idx is
342         when 0 =>
343             return setpoint;
344         when 1 =>
345             return feedback;
346         when 2 =>
347             return err_word;
348         when 3 =>
349             return pid_word;
350         when 4 =>
351             return pwm_word;
352         when others =>

```

```

353     word_idx_v := (C_DBG_ACCUM_WORDS - 1) - (idx - C_DBG_FIXED_WORDS);
354     accum_ext_v := resize(accum_word, C_DBG_ACCUM_WORDS*16);
355     return std_logic_vector(accum_ext_v((word_idx_v*16)+15 downto word_idx_v*16));
356 end case;
357 end function;
358
359 begin
360
361     bus_req <= bus_req_r;
362
363     setpoint_adc_s <= unsigned(setpoint_r(15 downto 4));
364
365     pid_error_s <= resize(signed('0' & std_logic_vector(setpoint_adc_s)), PID_ERROR_WIDTH)
366         - resize(signed('0' & feedback_r), PID_ERROR_WIDTH);
367
368     u_pid : entity work.My_PID
369         generic map (
370             ERROR_WIDTH => PID_ERROR_WIDTH,
371             COEFF_WIDTH => PID_COEFF_WIDTH,
372             FRAC_BITS => PID_COEFF_FRAC_BITS,
373             INTEGRATOR_SHIFT => PID_INTEGRATOR_SHIFT,
374             ACCUM_WIDTH => PID_ACCUM_WIDTH,
375             OUTPUT_WIDTH => PID_OUTPUT_WIDTH
376         )
377         port map (
378             clk => clk_sys,
379             reset_i => reset_i,
380             clear_accum_i => pid_clear_accum_r,
381             execute_i => pid_execute_r,
382             error_i => pid_error_s,
383             kp_i => kp_r,
384             ki_i => ki_r,
385             kd_i => kd_r,
386             accum_o => pid_accum_s,
387             pid_o => pid_output_s
388         );
389
390     pid_pwm_o <= pid_pwm_r;
391
392     p_pid_pwm : process(clk_sys)
393         variable divider_v : unsigned(15 downto 0);
394     begin
395         if rising_edge(clk_sys) then
396             if reset_i = '1' then
397                 pwm_count_r <= (others => '0');
398                 pwm_duty_latched_r <= (others => '0');
399                 pwm_div_count_r <= (others => '0');
400                 pid_pwm_r <= '0';
401             else
402                 divider_v := pwm_divider_r;
403
404                 if divider_v = 0 then
405                     divider_v := to_unsigned(1, divider_v'length);
406                 end if;
407
408                 if pwm_div_count_r >= (divider_v - 1) then
409                     pwm_div_count_r <= (others => '0');
410
411                     if pwm_count_r = 0 then

```

```

412         pwm_duty_latched_r <= pwm_duty_r;
413     end if;
414
415     if pwm_count_r < pwm_duty_latched_r then
416         pid_pwm_r <= '1';
417     else
418         pid_pwm_r <= '0';
419     end if;
420
421     pwm_count_r <= pwm_count_r + 1;
422     else
423         pwm_div_count_r <= pwm_div_count_r + 1;
424     end if;
425 end if;
426 end if;
427 end process;
428
429 p_update_timer : process(clk_sys)
430 begin
431     if rising_edge(clk_sys) then
432         if reset_i = '1' then
433             update_period_count_r <= 0;
434             update_tick_r <= '0';
435         else
436             update_tick_r <= '0';
437
438             if update_period_count_r = PID_UPDATE_CLKS - 1 then
439                 update_period_count_r <= 0;
440                 update_tick_r <= '1';
441             else
442                 update_period_count_r <= update_period_count_r + 1;
443             end if;
444         end if;
445     end if;
446 end process;
447
448 p_ctrl : process(clk_sys)
449 begin
450     if rising_edge(clk_sys) then
451         bus_req_r.req <= '0';
452         bus_req_r.we <= '0';
453         bus_req_r.addr <= (others => '0');
454         bus_req_r.wdata <= (others => '0');
455         pid_execute_r <= '0';
456         pid_clear_accum_r <= '0';
457
458         if reset_i = '1' then
459             state <= ST_INIT_WRITE;
460             read_index_r <= 0;
461             read_data_r <= (others => '0');
462             init_index_r <= 0;
463             update_pending_r <= '0';
464             pwm_enable_r <= '0';
465             pwm_divider_r <= to_unsigned(PWM_DIVIDER, pwm_divider_r'length);
466             setpoint_r <= (others => '0');
467             feedback_r <= (others => '0');
468             kp_r <= (others => '0');
469             ki_r <= (others => '0');
470             kd_r <= (others => '0');

```

```

471     pwm_duty_r      <= (others => '0');
472     dbg_index_r     <= 0;
473     dbg_setpoint_r  <= (others => '0');
474     dbg_feedback_r  <= (others => '0');
475     dbg_error_r     <= (others => '0');
476     dbg_pid_r       <= (others => '0');
477     dbg_pwm_r       <= (others => '0');
478 else
479     if (update_tick_r = '1') and (update_pending_r = '0') then
480         update_pending_r <= '1';
481     end if;
482
483     case state is
484         when ST_INIT_WRITE =>
485             bus_req_r.req  <= '1';
486             bus_req_r.we   <= '1';
487             bus_req_r.addr <= f_init_addr(init_index_r);
488             bus_req_r.wdata <= f_init_data(init_index_r);
489             state          <= ST_INIT_WAIT_ACK;
490
491         when ST_INIT_WAIT_ACK =>
492             bus_req_r.req  <= '1';
493             bus_req_r.we   <= '1';
494             bus_req_r.addr <= f_init_addr(init_index_r);
495             bus_req_r.wdata <= f_init_data(init_index_r);
496
497             if bus_rsp.ack = '1' then
498                 case init_index_r is
499                     when 0 =>
500                         kp_r <= unsigned(C_KP_INIT);
501                     when 1 =>
502                         ki_r <= unsigned(C_KI_INIT);
503                     when others =>
504                         kd_r <= unsigned(C_KD_INIT);
505                 end case;
506
507                 state <= ST_INIT_GAP;
508             end if;
509
510         when ST_INIT_GAP =>
511             if init_index_r = 2 then
512                 state <= ST_IDLE;
513             else
514                 init_index_r <= init_index_r + 1;
515                 state        <= ST_INIT_WRITE;
516             end if;
517
518         when ST_IDLE =>
519             if update_pending_r = '1' then
520                 update_pending_r <= '0';
521                 read_index_r     <= 0;
522                 state            <= ST_REQ_READ;
523             end if;
524
525         when ST_REQ_READ =>
526             bus_req_r.req  <= '1';
527             bus_req_r.we   <= '0';
528             bus_req_r.addr <= f_read_addr(read_index_r);
529             state          <= ST_WAIT_ACK;

```

```

530
531 when ST_WAIT_ACK =>
532     bus_req_r.req  <= '1';
533     bus_req_r.we   <= '0';
534     bus_req_r.addr <= f_read_addr(read_index_r);
535
536     if bus_rsp.ack = '1' then
537         read_data_r <= bus_rsp.rdata;
538         state <= ST_READ_CAPTURE;
539     end if;
540
541 when ST_READ_CAPTURE =>
542     case read_index_r is
543         when 0 =>
544             pwm_enable_r <= read_data_r(0);
545         when 1 =>
546             pwm_divider_r <= f_freq_ctrl_to_div(read_data_r);
547         when 2 =>
548             kp_r <= unsigned(read_data_r);
549         when 3 =>
550             ki_r <= unsigned(read_data_r);
551         when 4 =>
552             kd_r <= unsigned(read_data_r);
553         when 5 =>
554             feedback_r <= read_data_r;
555         when others =>
556             setpoint_r <= read_data_r;
557     end case;
558
559     state <= ST_REQ_GAP;
560
561 when ST_REQ_GAP =>
562     if read_index_r = C_NUM_READS - 1 then
563         if pwm_enable_r = '0' then
564             state <= ST_HOLD_DISABLED;
565         elsif ki_r = 0 then
566             state <= ST_CLEAR_ACCUM;
567         else
568             state <= ST_EXEC_PULSE;
569         end if;
570     else
571         read_index_r <= read_index_r + 1;
572         state <= ST_REQ_READ;
573     end if;
574
575 when ST_HOLD_DISABLED =>
576     pid_clear_accum_r <= '1';
577     pwm_duty_r <= (others => '0');
578     dbg_index_r <= 0;
579     dbg_setpoint_r <= std_logic_vector(resize(setpoint_adc_s, 16));
580     dbg_feedback_r <= feedback_r;
581     dbg_error_r <= std_logic_vector(resize(pid_error_s, 16));
582     dbg_pid_r <= (others => '0');
583     dbg_pwm_r <= (others => '0');
584     state <= ST_DBG_WRITE;
585
586 when ST_CLEAR_ACCUM =>
587     pid_clear_accum_r <= '1';
588     state <= ST_EXEC_PULSE;

```

```

589
590 when ST_EXEC_PULSE =>
591     pid_execute_r <= '1';
592     state <= ST_WAIT_PID_1;
593
594 when ST_WAIT_PID_1 =>
595     state <= ST_WAIT_PID_2;
596
597 when ST_WAIT_PID_2 =>
598     state <= ST_WAIT_PID_3;
599
600 when ST_WAIT_PID_3 =>
601     state <= ST_LATCH_PWM;
602
603 when ST_LATCH_PWM =>
604     pwm_duty_r <= f_sat_to_pwm(pid_output_s, PWM_COUNTER_WIDTH);
605     dbg_index_r <= 0;
606     dbg_setpoint_r <= std_logic_vector(resize(setpoint_adc_s, 16));
607     dbg_feedback_r <= feedback_r;
608     dbg_error_r <= std_logic_vector(resize(pid_error_s, 16));
609     dbg_pid_r <= std_logic_vector(resize(pid_output_s, 16));
610     dbg_pwm_r <= std_logic_vector(resize(f_sat_to_pwm(pid_output_s,
        PWM_COUNTER_WIDTH), 16));
611     state <= ST_DBG_WRITE;
612
613 when ST_DBG_WRITE =>
614     bus_req_r.req <= '1';
615     bus_req_r.we <= '1';
616     bus_req_r.addr <= f_dbg_addr(dbg_index_r);
617     bus_req_r.wdata <= f_dbg_data(
618         dbg_index_r,
619         dbg_setpoint_r,
620         dbg_feedback_r,
621         dbg_error_r,
622         dbg_pid_r,
623         dbg_pwm_r,
624         pid_accum_s
625     );
626     state <= ST_DBG_WAIT_ACK;
627
628 when ST_DBG_WAIT_ACK =>
629     bus_req_r.req <= '1';
630     bus_req_r.we <= '1';
631     bus_req_r.addr <= f_dbg_addr(dbg_index_r);
632     bus_req_r.wdata <= f_dbg_data(
633         dbg_index_r,
634         dbg_setpoint_r,
635         dbg_feedback_r,
636         dbg_error_r,
637         dbg_pid_r,
638         dbg_pwm_r,
639         pid_accum_s
640     );
641
642     if bus_rsp.ack = '1' then
643         state <= ST_DBG_GAP;
644     end if;
645
646 when ST_DBG_GAP =>

```

```

647         if dbg_index_r = C_DBG_WORDS_TOTAL - 1 then
648             state <= ST_IDLE;
649         else
650             dbg_index_r <= dbg_index_r + 1;
651             state <= ST_DBG_WRITE;
652         end if;
653
654         when others =>
655             state <= ST_IDLE;
656         end case;
657     end if;
658 end if;
659 end process;
660
661 end architecture rtl;

```

A.3 My_PID.vhd

```

1  =====
2  -- File: My_PID.vhd
3  --
4  -- Description:
5  --   Simple, reusable PID controller core for VHDL designs.
6  --
7  --   This module performs one PID calculation each time execute_i makes a
8  --   low-to-high transition. After that calculation, the module will not run
9  --   again until execute_i is deasserted and asserted again.
10 --
11 --   The implementation uses straightforward fixed-point integer math:
12 --
13 --   * error_i is a signed value
14 --   * kp_i, ki_i, and kd_i are unsigned positive coefficients
15 --   * coefficients are interpreted as:
16 --
17 --       real_gain = coefficient / 2^FRAC_BITS
18 --
19 --   Example with FRAC_BITS = 10:
20 --
21 --       coefficient value 1024 => gain = 1.0
22 --       coefficient value  512 => gain = 0.5
23 --       coefficient value  256 => gain = 0.25
24 --
25 --   Internally:
26 --       proportional = error * kp
27 --       integral     = (accumulator srl INTEGRATOR_SHIFT) * ki
28 --       derivative   = (error - previous_error) * kd
29 --
30 --   The accumulator is updated only when a valid execute pulse edge occurs.
31 --   The output is the sum of the three terms shifted right by FRAC_BITS.
32 --   Internally the arithmetic is pipelined across multiple clock cycles to
33 --   reduce FPGA timing pressure on smaller devices.
34 --
35 --   Features:
36 --   * Generic widths for error, coefficients, accumulator, and output
37 --   * Generic fixed-point fractional precision
38 --   * Generic integrator scaling shift to slow integral accumulation effect
39 --   * Active-high reset input

```

```

40 -- * Separate accumulator clear input
41 -- * One-shot execute behavior using execute_i edge detection
42 -- * Live-updateable P, I, and D coefficients
43 -- * Current accumulator state available on accum_o for wrapper-level debug
44 --
45 -- Notes:
46 -- * reset_i clears all internal state and forces pid_o to zero.
47 -- * clear_accum_i clears only the integrator accumulator.
48 -- * If clear_accum_i is asserted during an execute pulse, the accumulator
49 --   remains at zero after that step rather than being reloaded from the
50 --   current error.
51 -- * INTEGRATOR_SHIFT does not change how the accumulator is stored; it only
52 --   reduces the accumulator's contribution to the I term by shifting it
53 --   right before multiplication by ki_i.
54 -- * The derivative history is preserved across clear_accum_i. This keeps the
55 --   control simple and avoids surprising output discontinuities unless a full
56 --   reset is requested.
57 -- * Saturation is applied to the accumulator and output to avoid overflow.
58 --
59 -- Example:
60 --
61 -- u_pid : entity work.My_PID
62 --   generic map (
63 --     ERROR_WIDTH  => 16,
64 --     COEFF_WIDTH  => 16,
65 --     FRAC_BITS    => 10,
66 --     INTEGRATOR_SHIFT => 0,
67 --     ACCUM_WIDTH  => 24,
68 --     OUTPUT_WIDTH => 16
69 --   )
70 --   port map (
71 --     clk           => clk,
72 --     reset_i       => reset_i,
73 --     clear_accum_i => clear_accum_i,
74 --     execute_i     => sample_strobe,
75 --     error_i       => error_i,
76 --     kp_i          => kp_i,
77 --     ki_i          => ki_i,
78 --     kd_i          => kd_i,
79 --     accum_o       => open,
80 --     pid_o         => pid_o
81 --   );
82 =====
83
84 library ieee;
85 use ieee.std_logic_1164.all;
86 use ieee.numeric_std.all;
87
88 entity My_PID is
89   generic (
90     ERROR_WIDTH  : positive := 16;
91     COEFF_WIDTH  : positive := 16;
92     FRAC_BITS    : natural  := 10;
93     INTEGRATOR_SHIFT : natural := 0;
94     ACCUM_WIDTH  : positive := 24;
95     OUTPUT_WIDTH : positive := 16
96   );
97   port (
98     clk           : in  std_logic;

```

```

99     reset_i      : in  std_logic;
100     clear_accum_i : in  std_logic;
101     execute_i     : in  std_logic;
102     error_i       : in  signed(ERROR_WIDTH-1 downto 0);
103     kp_i          : in  unsigned(COEFF_WIDTH-1 downto 0);
104     ki_i          : in  unsigned(COEFF_WIDTH-1 downto 0);
105     kd_i          : in  unsigned(COEFF_WIDTH-1 downto 0);
106     accum_o       : out signed(ACCUM_WIDTH-1 downto 0);
107     pid_o         : out signed(OUTPUT_WIDTH-1 downto 0)
108 );
109 end entity My_PID;
110
111 architecture rtl of My_PID is
112
113     constant P_TERM_WIDTH : positive := ERROR_WIDTH + COEFF_WIDTH + 1;
114     constant I_TERM_WIDTH : positive := ACCUM_WIDTH + COEFF_WIDTH + 1;
115     constant D_TERM_WIDTH : positive := ACCUM_WIDTH + COEFF_WIDTH + 1;
116     constant TERM_WIDTH   : positive := ACCUM_WIDTH + COEFF_WIDTH + 1;
117     constant SUM_WIDTH    : positive := TERM_WIDTH + 2;
118
119     signal accum_r      : signed(ACCUM_WIDTH-1 downto 0) := (others => '0');
120     signal prev_error_r : signed(ERROR_WIDTH-1 downto 0) := (others => '0');
121     signal pid_r        : signed(OUTPUT_WIDTH-1 downto 0) := (others => '0');
122     signal execute_prev_r : std_logic := '0';
123     signal stage1_valid_r : std_logic := '0';
124     signal stage2_valid_r : std_logic := '0';
125
126     signal stage1_error_r      : signed(ERROR_WIDTH-1 downto 0) := (others => '0');
127     signal stage1_accum_r      : signed(ACCUM_WIDTH-1 downto 0) := (others => '0');
128     signal stage1_deriv_r      : signed(ACCUM_WIDTH-1 downto 0) := (others => '0');
129     signal stage1_kp_r         : unsigned(COEFF_WIDTH-1 downto 0) := (others => '0');
130     signal stage1_ki_r         : unsigned(COEFF_WIDTH-1 downto 0) := (others => '0');
131     signal stage1_kd_r         : unsigned(COEFF_WIDTH-1 downto 0) := (others => '0');
132     signal stage2_scaled_sum_r : signed(SUM_WIDTH-1 downto 0) := (others => '0');
133
134     function sat_resize_signed(
135         value_in      : signed;
136         target_size   : positive
137     ) return signed is
138         variable result_v : signed(target_size-1 downto 0);
139         variable fits_v   : boolean;
140     begin
141         if value_in'length <= target_size then
142             result_v := resize(value_in, target_size);
143         else
144             fits_v := true;
145
146             for idx in value_in'high downto target_size loop
147                 if value_in(idx) /= value_in(target_size-1) then
148                     fits_v := false;
149                 end if;
150             end loop;
151
152             if fits_v then
153                 result_v := resize(value_in, target_size);
154             elsif value_in(value_in'high) = '0' then
155                 result_v := (target_size-1 => '0', others => '1');
156             else
157                 result_v := (target_size-1 => '1', others => '0');

```

```

158     end if;
159 end if;
160
161 return result_v;
162 end function;
163
164 begin
165
166 process(clk)
167     variable error_ext_v      : signed(ACCUM_WIDTH-1 downto 0);
168     variable accum_next_v     : signed(ACCUM_WIDTH-1 downto 0);
169     variable deriv_v          : signed(ACCUM_WIDTH-1 downto 0);
170     variable execute_pulse_v  : boolean;
171     variable p_prod_v         : signed(P_TERM_WIDTH-1 downto 0);
172     variable i_prod_v         : signed(I_TERM_WIDTH-1 downto 0);
173     variable d_prod_v         : signed(D_TERM_WIDTH-1 downto 0);
174     variable p_term_v         : signed(TERM_WIDTH-1 downto 0);
175     variable i_term_v         : signed(TERM_WIDTH-1 downto 0);
176     variable d_term_v         : signed(TERM_WIDTH-1 downto 0);
177     variable sum_v            : signed(SUM_WIDTH-1 downto 0);
178     variable accum_i_v        : signed(ACCUM_WIDTH-1 downto 0);
179 begin
180     if rising_edge(clk) then
181         execute_pulse_v := (execute_i = '1') and (execute_prev_r = '0');
182
183         if reset_i = '1' then
184             accum_r      <= (others => '0');
185             prev_error_r  <= (others => '0');
186             pid_r         <= (others => '0');
187             execute_prev_r <= execute_i;
188             stage1_valid_r <= '0';
189             stage2_valid_r <= '0';
190             stage1_error_r <= (others => '0');
191             stage1_accum_r <= (others => '0');
192             stage1_deriv_r <= (others => '0');
193             stage1_kp_r    <= (others => '0');
194             stage1_ki_r    <= (others => '0');
195             stage1_kd_r    <= (others => '0');
196             stage2_scaled_sum_r <= (others => '0');
197
198         else
199             if stage2_valid_r = '1' then
200                 pid_r <= sat_resize_signed(stage2_scaled_sum_r, OUTPUT_WIDTH);
201             end if;
202
203             if stage1_valid_r = '1' then
204                 accum_i_v := shift_right(stage1_accum_r, INTEGRATOR_SHIFT);
205                 p_prod_v := stage1_error_r * signed('0' & stage1_kp_r);
206                 i_prod_v := accum_i_v * signed('0' & stage1_ki_r);
207                 d_prod_v := stage1_deriv_r * signed('0' & stage1_kd_r);
208
209                 p_term_v := resize(p_prod_v, TERM_WIDTH);
210                 i_term_v := resize(i_prod_v, TERM_WIDTH);
211                 d_term_v := resize(d_prod_v, TERM_WIDTH);
212
213                 sum_v := resize(p_term_v, SUM_WIDTH)
214                     + resize(i_term_v, SUM_WIDTH)
215                     + resize(d_term_v, SUM_WIDTH);
216

```

```

217         stage2_scaled_sum_r <= shift_right(sum_v, FRAC_BITS);
218         stage2_valid_r      <= '1';
219     else
220         stage2_valid_r <= '0';
221     end if;
222
223     if clear_accum_i = '1' then
224         accum_r <= (others => '0');
225     end if;
226
227     if execute_pulse_v then
228         error_ext_v := resize(error_i, ACCUM_WIDTH);
229         accum_next_v := sat_resize_signed(accum_r + error_ext_v, ACCUM_WIDTH);
230         deriv_v      := resize(error_i, ACCUM_WIDTH) - resize(prev_error_r, ACCUM_WIDTH);
231
232         if clear_accum_i = '1' then
233             accum_next_v := (others => '0');
234         end if;
235
236         accum_r      <= accum_next_v;
237         prev_error_r <= error_i;
238         stage1_error_r <= error_i;
239         stage1_accum_r <= accum_next_v;
240         stage1_deriv_r <= deriv_v;
241         stage1_kp_r    <= kp_i;
242         stage1_ki_r    <= ki_i;
243         stage1_kd_r    <= kd_i;
244         stage1_valid_r <= '1';
245     else
246         stage1_valid_r <= '0';
247     end if;
248
249     execute_prev_r <= execute_i;
250 end if;
251 end if;
252 end process;
253
254 accum_o <= accum_r;
255 pid_o <= pid_r;
256
257 end architecture rtl;

```

A.4 PWM_Module.vhd

```

1  =====
2  -- File: pwm_generic.vhd
3  --
4  -- Description:
5  --   General-purpose PWM generator for FPGA designs.
6  --
7  --   This module produces a pulse-width-modulated output using a free-running
8  --   PWM counter. The PWM resolution is set by COUNTER_WIDTH, and the effective
9  --   PWM frequency is reduced by CLK_DIVIDER. The duty cycle is provided as an
10 --   unsigned input and is latched only at the start of each PWM period to
11 --   prevent mid-cycle glitches.
12 --
13 -- Features:

```

```

14 -- * Configurable PWM resolution via COUNTER_WIDTH
15 -- * Configurable clock division via CLK_DIVIDER
16 -- * Glitch-free duty-cycle updates
17 -- * Enable input
18 -- * Optional active-high or active-low output polarity
19 --
20 -- Usage Notes:
21 -- * duty_i = 0                -> 0% duty cycle
22 -- * duty_i near maximum      -> near 100% duty cycle
23 -- * New duty values take effect only at the beginning of a PWM period
24 -- * When enable_i = '0', the PWM output is forced inactive and internal
25 --   counters are reset.
26 --
27 -- Frequency:
28 --   pwm_step_freq = f_clk / CLK_DIVIDER
29 --   pwm_freq      = f_clk / (CLK_DIVIDER * 2^COUNTER_WIDTH)
30 --
31 -- Example:
32 --   If f_clk = 25 MHz, CLK_DIVIDER = 10, and COUNTER_WIDTH = 8:
33 --
34 --       pwm_freq = 25e6 / (10 * 256)
35 --               ~= 9.77 kHz
36 --
37 -- Example Instantiation:
38 --
39 --   signal pwm_duty : unsigned(7 downto 0);
40 --   signal pwm_out  : std_logic;
41 --
42 --   u_pwm : entity work.pwm_generic
43 --     generic map (
44 --       COUNTER_WIDTH => 8,
45 --       CLK_DIVIDER   => 10,
46 --       ACTIVE_HIGH    => true
47 --     )
48 --     port map (
49 --       clk      => clk_sys,
50 --       reset_i  => reset_i,
51 --       enable_i => '1',
52 --       duty_i   => pwm_duty,
53 --       pwm_o    => pwm_out
54 --     );
55 --
56 --   -- Example duty settings
57 --   pwm_duty <= to_unsigned(64, 8); -- ~25%
58 --   pwm_duty <= to_unsigned(128, 8); -- ~50%
59 --   pwm_duty <= to_unsigned(192, 8); -- ~75%
60 --
61 -- Build Date:    2026-03-06
62 -- Last Updated:  2026-03-06
63 =====
64
65 library ieee;
66 use ieee.std_logic_1164.all;
67 use ieee.numeric_std.all;
68
69 entity pwm_generic is
70   generic (
71     COUNTER_WIDTH : positive := 8;
72     CLK_DIVIDER    : positive := 1;

```

```

73     ACTIVE_HIGH    : boolean := true
74 );
75 port (
76     clk           : in  std_logic;
77     reset_i       : in  std_logic;
78     enable_i      : in  std_logic;
79     duty_i        : in  unsigned(COUNTER_WIDTH-1 downto 0);
80     pwm_o         : out std_logic
81 );
82 end entity;
83
84 architecture rtl of pwm_generic is
85
86     signal pwm_count      : unsigned(COUNTER_WIDTH-1 downto 0) := (others => '0');
87     signal duty_latched   : unsigned(COUNTER_WIDTH-1 downto 0) := (others => '0');
88     signal pwm_int        : std_logic := '0';
89
90     signal div_count      : natural range 0 to CLK_DIVIDER-1 := 0;
91
92 begin
93
94     process(clk)
95         variable step_pwm_v : boolean;
96     begin
97         if rising_edge(clk) then
98             if reset_i = '1' then
99                 pwm_count      <= (others => '0');
100                duty_latched <= (others => '0');
101                pwm_int        <= '0';
102                div_count      <= 0;
103
104             else
105                 step_pwm_v := false;
106
107                 if enable_i = '1' then
108
109                     -----
110                     -- Clock divider
111                     -----
112                     if div_count = CLK_DIVIDER - 1 then
113                         div_count <= 0;
114                         step_pwm_v := true;
115                     else
116                         div_count <= div_count + 1;
117                     end if;
118
119                     -----
120                     -- Advance PWM engine
121                     -----
122                     if step_pwm_v then
123
124                         if pwm_count = 0 then
125                             duty_latched <= duty_i;
126                         end if;
127
128                         if pwm_count < duty_latched then
129                             pwm_int <= '1';
130                         else
131                             pwm_int <= '0';

```

```

132         end if;
133
134         pwm_count <= pwm_count + 1;
135
136         end if;
137
138         else
139             pwm_count <= (others => '0');
140             duty_latched <= duty_i;
141             pwm_int <= '0';
142             div_count <= 0;
143         end if;
144     end if;
145 end if;
146 end process;
147
148 pwm_o <= pwm_int when ACTIVE_HIGH else not pwm_int;
149
150 end architecture;

```

A.5 bus_pkg.vhd

```

1  -- =====
2  -- File: bus_pkg.vhd
3  --
4  -- Creator: Kevin Lyon / ChatGPT
5  -- Date Created: 18 March 2026
6  --
7  -- Description:
8  --   Common bus interface type definitions for the SPRAM client interface.
9  --
10 -- Notes:
11 --   I can't figure out mixed-direction record for entity ports in VHDL,
12 --   so the interface is split into:
13 --
14 --       t_bus_req : client -> Bus_Master
15 --       t_bus_rsp : Bus_Master -> client
16 --
17 -- =====
18
19 library ieee;
20 use ieee.std_logic_1164.all;
21
22 package bus_pkg is
23
24     -----
25     -- Client request bundle
26     -----
27     type t_bus_req is record
28         req    : std_logic;
29         we     : std_logic;
30         addr   : std_logic_vector(9 downto 0);
31         wdata  : std_logic_vector(15 downto 0);
32     end record;
33
34     -----
35     -- Client response bundle

```

```

36  -----
37  type t_bus_rsp is record
38      ack      : std_logic;
39      rdata    : std_logic_vector(15 downto 0);
40  end record;
41
42  end package;
43
44  package body bus_pkg is
45  end package body;

```

A.6 Bus_Master.vhd

```

1  -- =====
2  -- File: Bus_Master.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 04 March 2026
6  -- Last Updated: 22 April 2026
7  --
8  -- Description:
9  --   Prioritized SPRAM wrapper / arbitration layer for the MachX03D internal
10 --   SPRAM block.
11 --
12 --   Earlier revisions of this module implemented the complete UART command
13 --   protocol parser and memory execution engine. That functionality has now
14 --   been moved into the UART protocol controller
15 --   (uart_rs232_fifo_wrap_controller.vhd).
16 --
17 --   The Bus_Master module is now responsible only for:
18 --
19 --       1) Arbitrating access to SPRAM between multiple requesters
20 --       2) Performing the actual SPRAM read and write transactions
21 --       3) Returning completion acknowledgements to the requesting client
22 --
23 --   Each client presents a simple transaction interface:
24 --
25 --       req      : request memory transaction
26 --       ack      : transaction complete
27 --       we       : write enable (0 = read, 1 = write)
28 --       addr     : SPRAM word address
29 --       wdata    : data to write
30 --       rdata    : returned read data
31 --
32 --   Arbitration policy:
33 --
34 --       Fixed priority arbitration.
35 --       Client 0 has highest priority.
36 --       Client 1 has second priority.
37 --       Client 2 has third priority.
38 --       Client 3 has lowest priority.
39 --
40 --   Transaction behavior:
41 --
42 --       Read:
43 --         - Client asserts req with we = 0 and address valid
44 --         - Bus_Master performs synchronous SPRAM read

```

```

45 --      - rdata returned and ack asserted
46 --
47 --      Write:
48 --      - Client asserts req with we = 1 and address/data valid
49 --      - Bus_Master performs a two-cycle write sequence
50 --      - ack asserted after commit cycle
51 --
52 --      Addressing:
53 --
54 --      SPRAM is 1024 x 16 words
55 --      Only address bits [9:0] are used.
56 --
57 --
58 -- Revision History:
59 --
60 --      2026-03-04
61 --      - Initial implementation of UART command parser FSM.
62 --      - Added RX FIFO pop/capture mechanism using pop_pending.
63 --      - Implemented read command path and TX response logic.
64 --
65 --      2026-03-05
66 --      - Updated for custom FIFO in UART Wrapper.
67 --      - Added intermediate FSM states to safely capture address and
68 --      data bytes before using them in logic decisions.
69 --      - Implemented addr16_next staging register to prevent stale
70 --      address slicing during address construction.
71 --      - Corrected bug where the low data byte could be written
72 --      with a stale value due to signal update timing.
73 --      - Final fix: mem_din assembled directly as
74 --      wdata(15 downto 8) & rx_byte
75 --      ensuring correct word writes to SPRAM.
76 --
77 --      2026-03-09
78 --      - Reworked protocol to packet-based framing using LEN, CHKSUM, and CR.
79 --      - Added buffered packet receiver with explicit POP / CAPTURE states.
80 --      - Added checksum validation before command execution.
81 --      - Added dedicated TX packet FSM for formatted replies.
82 --      - Added reusable checksum function for both RX and TX packet handling.
83 --      - Rewritten in a more behavioral / readable style.
84 --      - Updated write path to use a two-cycle write sequence:
85 --      RX_WRITE_SETUP -> present address/data
86 --      RX_WRITE_COMMIT -> pulse mem_en/mem_we
87 --      to improve SPRAM write robustness.
88 --
89 --      2026-03-10
90 --      - Changed RX checksum validation to a running XOR accumulated as
91 --      bytes are captured.
92 --      - This avoids post-buffer checksum timing issues where the final
93 --      payload byte could be missing from the computed checksum.
94 --
95 --      2026-03-11
96 --      - Added TX_PREP state in transmit FSM.
97 --      - Outgoing checksum and LEN are now computed one cycle after tx_start
98 --      so the response payload and payload length are stable.
99 --      - Fixes stale TX checksum issue that could omit the final payload byte
100 --      or use old payload contents.
101 --
102 --      2026-03-13
103 --      - Added COUNT field to both read and write command formats.

```

```

104 --      - Enabled sequential burst reads and writes to SPRAM.
105 --      - Separated packet reception (RX FSM) from memory execution (EXEC FSM).
106 --      - Introduced request queue handshake using req_valid / req_take.
107 --      - Added simple arbitration scaffold for future prioritized bus access.
108 --      - Changed TX checksum generation to a running XOR accumulated as
109 --        payload bytes are transmitted.
110 --      - This fixes the stale TX checksum issue where the final payload byte
111 --        was omitted from the returned checksum.
112 --
113 -- 2026-03-16
114 --      - Major architectural refactor.
115 --      - UART protocol parsing and packet execution moved to
116 --        uart_rs232_fifo_wrap_controller.vhd.
117 --      - Bus_Master redesigned as a reusable SPRAM arbitration wrapper.
118 --      - Introduced simple req/ack client transaction interface.
119 --      - Added fixed-priority multi-client arbitration scaffold.
120 --
121 -- 2026-03-18
122 --      - Refactored the flat client req/ack signal interface into structured
123 --        request and response records defined in bus_pkg.vhd.
124 --      - Updated client 0 and client 1 ports to use t_bus_req and t_bus_rsp.
125 --      - No intended functional change to arbitration or SPRAM timing.
126 --      - Added per-client request-consumed flags so each asserted req is
127 --        serviced only once until that client drops req low.
128 --      - Removed the separate re-arm process after discovering that the
129 --        request-consumed flags were being driven from two clocked processes.
130 --      - Moved both flag set and flag clear behavior into the main bus FSM
131 --        process so each flag is implemented as a single clean register.
132 --      - This prevents duplicate transactions from a held req without
133 --        stalling the main bus FSM and without creating multi-driver logic.
134 --
135 -- 2026-03-20
136 --      - Removed the per-client req_seen flags.
137 --      - Updated the final read and write FSM states to hold until the active
138 --        client's req line returns low before allowing the bus FSM to return
139 --        to BUS_IDLE.
140 --      - This restores one-transaction-per-request behavior using explicit
141 --        request deassertion rather than request-consumed latches.
142 --      - Added dedicated post-ack wait states so ack is asserted for only a
143 --        single clock cycle, after which the FSM waits for the active client's
144 --        req line to return low before returning to BUS_IDLE.
145 --
146 -- 2026-04-07
147 --      - Added a third client port for ADC access.
148 --      - Updated arbitration to fixed priority:
149 --        Client 0 = PWM (highest)
150 --        Client 1 = ADC (middle)
151 --        Client 2 = UART (lowest)
152 --      - No intended change to SPRAM transaction timing.
153 --
154 -- 2026-04-22
155 --      - Added a fourth client port for the PID wrapper.
156 --      - Updated arbitration priority to:
157 --        Client 0 = PWM
158 --        Client 1 = PID
159 --        Client 2 = ADC
160 --        Client 3 = UART
161 --      - UART remains the lowest-priority bus client.
162 --

```

```

163 -- =====
164
165 library ieee;
166 use ieee.std_logic_1164.all;
167 use ieee.numeric_std.all;
168
169 library work;
170 use work.bus_pkg.all;
171
172 entity Bus_Master is
173 port (
174     clk_sys : in  std_logic;
175     reset_i : in  std_logic;
176
177     -----
178     -- Client 0 PWM (highest priority)
179     -----
180     c0_req : in  t_bus_req;
181     c0_rsp : out t_bus_rsp;
182
183     -----
184     -- Client 1 PID (second priority)
185     -----
186     c1_req : in  t_bus_req;
187     c1_rsp : out t_bus_rsp;
188
189     -----
190     -- Client 2 ADC (third priority)
191     -----
192     c2_req : in  t_bus_req;
193     c2_rsp : out t_bus_rsp;
194
195     -----
196     -- Client 3 RS232 (lowest priority)
197     -----
198     c3_req : in  t_bus_req;
199     c3_rsp : out t_bus_rsp;
200
201     -----
202     -- SPRAM interface
203     -----
204     mem_addr : out std_logic_vector(9 downto 0);
205     mem_din  : out std_logic_vector(15 downto 0);
206     mem_dout : in  std_logic_vector(15 downto 0);
207     mem_we   : out std_logic;
208     mem_en   : out std_logic
209 );
210 end entity;
211
212 architecture behavioral of Bus_Master is
213
214     -----
215     -- Arbiter / transaction FSM
216     -----
217     type t_bus_state is (
218         BUS_IDLE,
219         BUS_READ_SETUP,
220         BUS_READ_RETURN,
221         BUS_READ_WAIT_REQ_LOW,

```

```

222     BUS_WRITE_SETUP ,
223     BUS_WRITE_COMMIT ,
224     BUS_WRITE_WAIT_REQ_LOW
225 );
226
227 type t_client_sel is (
228     SEL_NONE ,
229     SEL_C0 ,
230     SEL_C1 ,
231     SEL_C2 ,
232     SEL_C3
233 );
234
235 signal bus_state      : t_bus_state := BUS_IDLE;
236 signal active_client  : t_client_sel := SEL_NONE;
237
238 signal active_we      : std_logic := '0';
239 signal active_addr    : std_logic_vector(9 downto 0) := (others => '0');
240 signal active_wdata   : std_logic_vector(15 downto 0) := (others => '0');
241
242 signal c0_rsp_r       : t_bus_rsp := (
243     ack  => '0',
244     rdata => (others => '0')
245 );
246
247 signal c1_rsp_r       : t_bus_rsp := (
248     ack  => '0',
249     rdata => (others => '0')
250 );
251
252 signal c2_rsp_r       : t_bus_rsp := (
253     ack  => '0',
254     rdata => (others => '0')
255 );
256
257 signal c3_rsp_r       : t_bus_rsp := (
258     ack  => '0',
259     rdata => (others => '0')
260 );
261
262 begin
263
264     c0_rsp <= c0_rsp_r;
265     c1_rsp <= c1_rsp_r;
266     c2_rsp <= c2_rsp_r;
267     c3_rsp <= c3_rsp_r;
268
269     -----
270     -- Arbitration / SPRAM transaction engine
271     -----
272     p_bus_fsm : process(clk_sys)
273     begin
274         if rising_edge(clk_sys) then
275
276             -- defaults every cycle
277             c0_rsp_r.ack <= '0';
278             c1_rsp_r.ack <= '0';
279             c2_rsp_r.ack <= '0';
280             c3_rsp_r.ack <= '0';

```

```

281
282 mem_en <= '0';
283 mem_we <= '0';
284
285 if reset_i = '1' then
286     bus_state      <= BUS_IDLE;
287     active_client  <= SEL_NONE;
288     active_we      <= '0';
289     active_addr    <= (others => '0');
290     active_wdata   <= (others => '0');
291
292     c0_rsp_r.ack    <= '0';
293     c0_rsp_r.rdata  <= (others => '0');
294     c1_rsp_r.ack    <= '0';
295     c1_rsp_r.rdata  <= (others => '0');
296     c2_rsp_r.ack    <= '0';
297     c2_rsp_r.rdata  <= (others => '0');
298     c3_rsp_r.ack    <= '0';
299     c3_rsp_r.rdata  <= (others => '0');
300
301     mem_addr       <= (others => '0');
302     mem_din        <= (others => '0');
303
304 else
305
306     case bus_state is
307
308         -----
309         -- Choose next requester by fixed priority
310         -----
311
312         when BUS_IDLE =>
313             if c0_req.req = '1' then
314                 active_client <= SEL_C0;
315                 active_we     <= c0_req.we;
316                 active_addr   <= c0_req.addr;
317                 active_wdata  <= c0_req.wdata;
318
319                 if c0_req.we = '1' then
320                     bus_state <= BUS_WRITE_SETUP;
321                 else
322                     bus_state <= BUS_READ_SETUP;
323                 end if;
324
325             elsif c1_req.req = '1' then
326                 active_client <= SEL_C1;
327                 active_we     <= c1_req.we;
328                 active_addr   <= c1_req.addr;
329                 active_wdata  <= c1_req.wdata;
330
331                 if c1_req.we = '1' then
332                     bus_state <= BUS_WRITE_SETUP;
333                 else
334                     bus_state <= BUS_READ_SETUP;
335                 end if;
336
337             elsif c2_req.req = '1' then
338                 active_client <= SEL_C2;
339                 active_we     <= c2_req.we;
340                 active_addr   <= c2_req.addr;

```

```

340         active_wdata <= c2_req.wdata;
341
342         if c2_req.we = '1' then
343             bus_state <= BUS_WRITE_SETUP;
344         else
345             bus_state <= BUS_READ_SETUP;
346         end if;
347
348     elsif c3_req.req = '1' then
349         active_client <= SEL_C3;
350         active_we <= c3_req.we;
351         active_addr <= c3_req.addr;
352         active_wdata <= c3_req.wdata;
353
354         if c3_req.we = '1' then
355             bus_state <= BUS_WRITE_SETUP;
356         else
357             bus_state <= BUS_READ_SETUP;
358         end if;
359
360     else
361         bus_state <= BUS_IDLE;
362     end if;
363
364     -----
365     -- Read: present address and enable SPRAM
366     -----
367     when BUS_READ_SETUP =>
368         mem_addr <= active_addr;
369         mem_en <= '1';
370         mem_we <= '0';
371         bus_state <= BUS_READ_RETURN;
372
373     -----
374     -- Read: data now valid from synchronous SPRAM
375     -----
376     when BUS_READ_RETURN =>
377         if active_client = SEL_C0 then
378             c0_rsp_r.rdata <= mem_dout;
379             c0_rsp_r.ack <= '1';
380             bus_state <= BUS_READ_WAIT_REQ_LOW;
381
382         elsif active_client = SEL_C1 then
383             c1_rsp_r.rdata <= mem_dout;
384             c1_rsp_r.ack <= '1';
385             bus_state <= BUS_READ_WAIT_REQ_LOW;
386
387         elsif active_client = SEL_C2 then
388             c2_rsp_r.rdata <= mem_dout;
389             c2_rsp_r.ack <= '1';
390             bus_state <= BUS_READ_WAIT_REQ_LOW;
391
392         elsif active_client = SEL_C3 then
393             c3_rsp_r.rdata <= mem_dout;
394             c3_rsp_r.ack <= '1';
395             bus_state <= BUS_READ_WAIT_REQ_LOW;
396
397     else
398         active_client <= SEL_NONE;

```

```

399         bus_state      <= BUS_IDLE;
400     end if;
401
402     -----
403     -- Read complete: wait for requester to release req before re-arming
404     -----
405     when BUS_READ_WAIT_REQ_LOW =>
406         if active_client = SEL_C0 then
407             if c0_req.req = '0' then
408                 active_client <= SEL_NONE;
409                 bus_state      <= BUS_IDLE;
410             else
411                 bus_state      <= BUS_READ_WAIT_REQ_LOW;
412             end if;
413
414         elsif active_client = SEL_C1 then
415             if c1_req.req = '0' then
416                 active_client <= SEL_NONE;
417                 bus_state      <= BUS_IDLE;
418             else
419                 bus_state      <= BUS_READ_WAIT_REQ_LOW;
420             end if;
421
422         elsif active_client = SEL_C2 then
423             if c2_req.req = '0' then
424                 active_client <= SEL_NONE;
425                 bus_state      <= BUS_IDLE;
426             else
427                 bus_state      <= BUS_READ_WAIT_REQ_LOW;
428             end if;
429
430         elsif active_client = SEL_C3 then
431             if c3_req.req = '0' then
432                 active_client <= SEL_NONE;
433                 bus_state      <= BUS_IDLE;
434             else
435                 bus_state      <= BUS_READ_WAIT_REQ_LOW;
436             end if;
437
438         else
439             active_client <= SEL_NONE;
440             bus_state      <= BUS_IDLE;
441         end if;
442
443     -----
444     -- Write: setup address/data for one full cycle
445     -----
446     when BUS_WRITE_SETUP =>
447         mem_addr <= active_addr;
448         mem_din  <= active_wdata;
449         mem_we    <= '0';
450         mem_en    <= '0';
451         bus_state <= BUS_WRITE_COMMIT;
452
453     -----
454     -- Write: commit write pulse and issue single-cycle ack
455     -----
456     when BUS_WRITE_COMMIT =>
457         mem_addr <= active_addr;

```

```

458     mem_din    <= active_wdata;
459     mem_en     <= '1';
460     mem_we     <= '1';
461
462     if active_client = SEL_C0 then
463         c0_rsp_r.ack <= '1';
464         bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
465
466     elsif active_client = SEL_C1 then
467         c1_rsp_r.ack <= '1';
468         bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
469
470     elsif active_client = SEL_C2 then
471         c2_rsp_r.ack <= '1';
472         bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
473
474     elsif active_client = SEL_C3 then
475         c3_rsp_r.ack <= '1';
476         bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
477
478     else
479         active_client <= SEL_NONE;
480         bus_state     <= BUS_IDLE;
481     end if;
482
483     -----
484     -- Write complete: wait for requester to release req before re-arming
485     -----
486     when BUS_WRITE_WAIT_REQ_LOW =>
487         if active_client = SEL_C0 then
488             if c0_req.req = '0' then
489                 active_client <= SEL_NONE;
490                 bus_state     <= BUS_IDLE;
491             else
492                 bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
493             end if;
494
495         elsif active_client = SEL_C1 then
496             if c1_req.req = '0' then
497                 active_client <= SEL_NONE;
498                 bus_state     <= BUS_IDLE;
499             else
500                 bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
501             end if;
502
503         elsif active_client = SEL_C2 then
504             if c2_req.req = '0' then
505                 active_client <= SEL_NONE;
506                 bus_state     <= BUS_IDLE;
507             else
508                 bus_state    <= BUS_WRITE_WAIT_REQ_LOW;
509             end if;
510
511         elsif active_client = SEL_C3 then
512             if c3_req.req = '0' then
513                 active_client <= SEL_NONE;
514                 bus_state     <= BUS_IDLE;
515             else
516                 bus_state    <= BUS_WRITE_WAIT_REQ_LOW;

```

```

517         end if;
518
519     else
520         active_client <= SEL_NONE;
521         bus_state      <= BUS_IDLE;
522     end if;
523
524     when others =>
525         bus_state <= BUS_IDLE;
526     end case;
527 end if;
528 end if;
529 end process;
530
531 end architecture;

```

A.7 pwm_spram_client.vhd

```

1  -- =====
2  -- File: pwm_spram_client.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 24 March 2026
6  -- Last Updated: 03 May 2026
7  --
8  -- Description:
9  --   Memory-mapped multi-channel PWM client wrapper for the shared SPRAM /
10 --   Bus_Master architecture.
11 --
12 --   Current register usage:
13 --       0x0100 : global enable in bit 0 for PWM_0..PWM_3
14 --       0x0101 : PWM frequency control, not blink period
15 --           0 => fastest PWM
16 --           65535 => slowest PWM in the bounded configured range
17 --       0x0103 : PWM_0 duty
18 --       0x0104 : PWM_1 duty
19 --       0x0105 : PWM_2 duty
20 --       0x0106 : PWM_3 duty
21 --
22 --   The legacy blink-period register at 0x0101 now controls PWM switching
23 --   frequency over a bounded range. 0x0102 remains unused. There is no blink
24 --   envelope in this module.
25 --
26 --   Enable behavior:
27 --       reg_enable(0) = '1' enables all four PWM outputs
28 --       reg_enable(0) = '0' disables all four PWM outputs
29 --
30 -- Revision History:
31 --   2026-04-07
32 --       - Reduced PWM channels from 8 to 4 (PWM_0 through PWM_3 only).
33 --       - Removed PWM_4 through PWM_7 from all logic, registers, and outputs.
34 --
35 --   2026-04-22
36 --       - Removed blink behavior from the LED PWM client.
37 --       - At that revision, 0x0101 and 0x0102 were ignored here.
38 --       - PWM outputs are driven directly from the duty registers at 0x0103+.
39 --       - Changed 0x0100 bit 0 to act as a global enable for all four LEDs.

```

```

40 --
41 -- 2026-05-03
42 -- - Reassigned 0x0101 from ignored legacy blink-period storage to a live
43 --   bounded 16-bit PWM frequency control.
44 -- - Replaced fixed-divider pwm_generic instances with local PWM engines
45 --   that share the runtime frequency control.
46 -- - Duty registers are scaled from the 16-bit LabVIEW words into the
47 --   configured PWM width by taking the most significant bits.
48 --
49 -- =====
50
51 library ieee;
52 use ieee.std_logic_1164.all;
53 use ieee.numeric_std.all;
54
55 library work;
56 use work.bus_pkg.all;
57
58 entity pwm_spram_client is
59   generic (
60     CLK_FREQ_HZ      : integer := 24930000;
61     PWM_WIDTH        : integer := 16;
62     PWM_DIVIDER       : integer := 1;
63     PWM_FREQ_CTRL_MAX_DIV : integer := 6;
64     REG_BASE_ADDR     : integer := 16#0100#;
65     POLL_CYCLES      : integer := 25000
66   );
67   port (
68     clk_sys : in  std_logic;
69     reset_i : in  std_logic;
70
71     bus_req : out t_bus_req;
72     bus_rsp : in  t_bus_rsp;
73
74     PWM_0 : out std_logic;
75     PWM_1 : out std_logic;
76     PWM_2 : out std_logic;
77     PWM_3 : out std_logic
78   );
79 end entity;
80
81 architecture rtl of pwm_spram_client is
82
83   -----
84   -- Register map offsets
85   -----
86   constant C_ADDR_EN      : integer := REG_BASE_ADDR + 0;
87   constant C_ADDR_FREQ    : integer := REG_BASE_ADDR + 1;
88   constant C_ADDR_PW      : integer := REG_BASE_ADDR + 2;
89   constant C_ADDR_PWM0_DC : integer := REG_BASE_ADDR + 3;
90   constant C_NUM_REGS     : integer := 7;
91
92   -----
93   -- Types
94   -----
95   type t_state is (
96     ST_POLL,
97     ST_REQ,
98     ST_WAIT_ACK,

```

```

99     ST_READ_CAPTURE,
100     ST_NEXT_REG
101 );
102
103 type t_duty_array is array (0 to 3) of unsigned(PWM_WIDTH-1 downto 0);
104 type t_pwm_logic_array is array (0 to 3) of std_logic;
105
106 -----
107 -- Bus request register
108 -----
109 signal bus_req_r : t_bus_req := (
110     req    => '0',
111     we     => '0',
112     addr   => (others => '0'),
113     wdata  => (others => '0')
114 );
115
116 -----
117 -- FSM / polling
118 -----
119 signal state      : t_state := ST_POLL;
120 signal poll_count : integer range 0 to POLL_CYCLES-1 := POLL_CYCLES-1;
121 signal reg_index  : integer range 0 to C_NUM_REGS-1 := 0;
122 signal curr_addr  : std_logic_vector(9 downto 0) := (others => '0');
123 signal read_data_r : std_logic_vector(15 downto 0) := (others => '0');
124
125 -----
126 -- Local shadow registers
127 -----
128 signal reg_enable      : std_logic_vector(7 downto 0) := (others => '0');
129 signal reg_freq_ctrl   : std_logic_vector(15 downto 0) := (others => '0');
130 signal pwm_freq_div_r  : unsigned(15 downto 0) := to_unsigned(PWM_DIVIDER, 16);
131 signal pwm_duty        : t_duty_array := (others => (others => '0'));
132
133 -----
134 -- Per-channel enables / outputs
135 -----
136 signal pwm_enable : std_logic_vector(3 downto 0) := (others => '0');
137 signal pwm_raw    : t_pwm_logic_array := (others => '0');
138 signal pwm_count  : unsigned(PWM_WIDTH-1 downto 0) := (others => '0');
139 signal pwm_duty_latched : t_duty_array := (others => (others => '0'));
140 signal pwm_div_count : unsigned(15 downto 0) := (others => '0');
141
142 function f_scale_duty(
143     value_in : std_logic_vector(15 downto 0)
144 ) return unsigned is
145     variable result_v : unsigned(PWM_WIDTH-1 downto 0);
146 begin
147     if PWM_WIDTH <= 16 then
148         result_v := unsigned(value_in(15 downto 16-PWM_WIDTH));
149     else
150         result_v := resize(unsigned(value_in), PWM_WIDTH);
151     end if;
152
153     return result_v;
154 end function;
155
156 function f_freq_ctrl_to_div(
157     value_in : std_logic_vector(15 downto 0)

```

```

158 ) return unsigned is
159     variable ctrl_v : integer;
160     variable div_v   : integer;
161 begin
162     ctrl_v := to_integer(unsigned(value_in));
163
164     if PWM_FREQ_CTRL_MAX_DIV <= 1 then
165         div_v := 1;
166     else
167         div_v := 1 + ((ctrl_v * (PWM_FREQ_CTRL_MAX_DIV - 1) + 32767) / 65535);
168     end if;
169
170     return to_unsigned(div_v, 16);
171 end function;
172
173 begin
174
175     bus_req <= bus_req_r;
176
177     curr_addr <= std_logic_vector(to_unsigned(REG_BASE_ADDR + reg_index, 10));
178
179     -----
180     -- Polling FSM
181     -----
182     p_client_fsm : process(clk_sys)
183     begin
184         if rising_edge(clk_sys) then
185             if reset_i = '1' then
186                 state      <= ST_POLL;
187                 poll_count <= POLL_CYCLES-1;
188                 reg_index  <= 0;
189                 read_data_r <= (others => '0');
190
191                 bus_req_r.req <= '0';
192                 bus_req_r.we  <= '0';
193
194             else
195                 case state is
196
197                     when ST_POLL =>
198                         bus_req_r.req <= '0';
199
200                         reg_index <= 0;
201
202                         if poll_count = POLL_CYCLES-1 then
203                             poll_count <= 0;
204                             state      <= ST_REQ;
205                         else
206                             poll_count <= poll_count + 1;
207                         end if;
208
209                     when ST_REQ =>
210                         bus_req_r.req <= '1';
211                         bus_req_r.we  <= '0';
212                         bus_req_r.addr <= curr_addr;
213
214                         state <= ST_WAIT_ACK;
215
216                     when ST_WAIT_ACK =>

```

```

217         if bus_rsp.ack = '1' then
218             read_data_r <= bus_rsp.rdata;
219             bus_req_r.req <= '0';
220             state          <= ST_READ_CAPTURE;
221         end if;
222
223     when ST_READ_CAPTURE =>
224         case reg_index is
225             when 0 => reg_enable <= read_data_r(7 downto 0);
226             when 1 =>
227                 reg_freq_ctrl <= read_data_r;
228                 pwm_freq_div_r <= f_freq_ctrl_to_div(read_data_r);
229             when 2 => null;
230             when 3 => pwm_duty(0) <= f_scale_duty(read_data_r);
231             when 4 => pwm_duty(1) <= f_scale_duty(read_data_r);
232             when 5 => pwm_duty(2) <= f_scale_duty(read_data_r);
233             when 6 => pwm_duty(3) <= f_scale_duty(read_data_r);
234             when others => null;
235         end case;
236
237         state <= ST_NEXT_REG;
238
239     when ST_NEXT_REG =>
240         if reg_index < (C_NUM_REGS - 1) then
241             reg_index <= reg_index + 1;
242             state      <= ST_REQ;
243         else
244             reg_index <= 0;
245             state      <= ST_POLL;
246         end if;
247
248     end case;
249 end if;
250 end if;
251 end process;
252
253 -----
254 -- Enable mapping
255 -----
256 pwm_enable(0) <= reg_enable(0);
257 pwm_enable(1) <= reg_enable(0);
258 pwm_enable(2) <= reg_enable(0);
259 pwm_enable(3) <= reg_enable(0);
260
261 -----
262 -- Runtime-divider PWM generators
263 -----
264 p_pwm : process(clk_sys)
265     variable divider_v : unsigned(15 downto 0);
266 begin
267     if rising_edge(clk_sys) then
268         if reset_i = '1' then
269             pwm_count      <= (others => '0');
270             pwm_duty_latched <= (others => (others => '0'));
271             pwm_div_count   <= (others => '0');
272             pwm_raw         <= (others => '0');
273         else
274             divider_v := pwm_freq_div_r;
275

```

```

276     if divider_v = 0 then
277         divider_v := to_unsigned(1, divider_v'length);
278     end if;
279
280     if pwm_div_count >= (divider_v - 1) then
281         pwm_div_count <= (others => '0');
282
283         if pwm_count = 0 then
284             pwm_duty_latched <= pwm_duty;
285         end if;
286
287         for i in 0 to 3 loop
288             if (pwm_enable(i) = '1') and (pwm_count < pwm_duty_latched(i)) then
289                 pwm_raw(i) <= '1';
290             else
291                 pwm_raw(i) <= '0';
292             end if;
293         end loop;
294
295         pwm_count <= pwm_count + 1;
296     else
297         pwm_div_count <= pwm_div_count + 1;
298     end if;
299 end if;
300 end if;
301 end process;
302
303 -----
304 -- Outputs
305 -----
306 PWM_0 <= pwm_raw(0);
307 PWM_1 <= pwm_raw(1);
308 PWM_2 <= pwm_raw(2);
309 PWM_3 <= pwm_raw(3);
310
311 end architecture;

```

A.8 adc_spram_client.vhd

```

1  -- =====
2  -- File: adc_spram_client.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 24 March 2026
6  -- Last Updated: 19 April 2026
7  --
8  -- Description:
9  --   Memory-mapped ADC client wrapper for the shared SPRAM / Bus_Master
10 --   architecture.
11 --
12 --   This module periodically starts a conversion on the AD7928 SPI ADC core,
13 --   captures each valid 12-bit sample, and writes that sample into a selected
14 --   SPRAM register through the Bus_Master client interface.
15 --
16 --   The intent is to make ADC feedback available as a memory-mapped register
17 --   so that other modules, such as a PID controller or UART interface, may
18 --   read the latest sampled value without directly interacting with the ADC

```

```

19 -- timing or SPI protocol.
20 --
21 -- Current default mapping:
22 --
23 --     SPRAM address ADC_BASE_ADDR + channel_number
24 --     Latest sample for each channel
25 --
26 -- The 12-bit ADC result is zero-extended into the 16-bit SPRAM data word:
27 --
28 --     bus_req.wdata <= "0000" & adc_result
29 --
30 -- Bus behavior:
31 --     - Write-only client of Bus_Master
32 --     - Starts a burst ADC sweep once per programmable sweep period
33 --     - Waits for adc_data_valid
34 --     - Latches most recent valid sample
35 --     - Issues one write request to the selected SPRAM address
36 --     - Waits for ack
37 --     - Deasserts req cleanly before returning to idle
38 --
39 -- ADC behavior:
40 --     - Instantiates ad7928_spi_master from AD7928_Core.vhd
41 --     - Uses programmable start-to-start period between complete sweeps
42 --     - Cycles through all eight AD7928 channels in a burst
43 --     - Uses straight-binary coding and 0..2*REFIN range by default
44 --
45 -- Notes:
46 --     - The AD7928 returns conversion data from the previous frame, so the
47 --     first completed frame primes the pipeline and does not produce a valid
48 --     stored sample. This is already handled by ad7928_spi_master via the
49 --     data_valid_o output.
50 --     - If a new ADC sample arrives while a previous bus write is still pending,
51 --     the wrapper retains only the most recent sample. For the expected
52 --     sample and bus rates in this project, this is acceptable.
53 --
54 -- Revision History:
55 --
56 -- 2026-03-24
57 --     - Initial version.
58 --     - Added periodic AD7928 conversion control.
59 --     - Added sample capture and Bus_Master write interface.
60 --     - Added zero-extension of 12-bit ADC sample to 16-bit SPRAM word.
61 --
62 -- 2026-04-17
63 --     - Updated from fixed single-channel sampling to round-robin sampling of
64 --     all eight AD7928 channels.
65 --     - Added sequential SPRAM storage so each returned channel sample is
66 --     written to ADC_BASE_ADDR + channel_id.
67 --     - Updated the scheduler so one complete 8-channel sweep is acquired and
68 --     written to SPRAM once per SAMPLE_PERIOD_CLKS start-to-start interval.
69 --
70 -- 2026-04-19
71 --     - Updated the AD7928 control configuration to drive RANGE=0 so the ADC
72 --     command format matches the class reference slide's 0V to 2*REFIN mode.
73 --
74 -- =====
75
76 library ieee;
77 use ieee.std_logic_1164.all;

```

```

78 use ieee.numeric_std.all;
79
80 library work;
81 use work.bus_pkg.all;
82
83 entity adc_spram_client is
84   generic (
85     CLK_FREQ_HZ      : integer := 24930000;
86     ADC_CLK_DIV      : positive := 5;
87     SAMPLE_PERIOD_CLKS : integer := 24930;
88     ADC_BASE_ADDR     : integer := 16#0200#
89   );
90   port (
91     clk_sys : in  std_logic;
92     reset_i : in  std_logic;
93
94     -- Client bus to Bus_Master
95     bus_req : out t_bus_req;
96     bus_rsp : in  t_bus_rsp;
97
98     -- AD7928 pins
99     adc_cs_n_o : out std_logic;
100    adc_sclk_o : out std_logic;
101    adc_din_o  : out std_logic;
102    adc_dout_i : in  std_logic
103  );
104 end entity;
105
106 architecture rtl of adc_spram_client is
107
108   -----
109   -- Bus request register
110   -----
111   signal bus_req_r : t_bus_req := (
112     req  => '0',
113     we   => '0',
114     addr => (others => '0'),
115     wdata => (others => '0')
116   );
117
118   -----
119   -- ADC core interface
120   -----
121   signal adc_start      : std_logic := '0';
122   signal adc_busy       : std_logic := '0';
123   signal adc_data_valid : std_logic := '0';
124   signal adc_primed     : std_logic := '0';
125   signal adc_result     : std_logic_vector(11 downto 0) := (others => '0');
126   signal adc_result_chan : std_logic_vector(2 downto 0) := (others => '0');
127   signal adc_raw_word   : std_logic_vector(15 downto 0) := (others => '0');
128   signal adc_next_chan  : std_logic_vector(2 downto 0) := (others => '0');
129   signal sweep_start_pulse : std_logic := '0';
130   signal discard_next_sample : std_logic := '0';
131
132   -----
133   -- Sweep period timer
134   -----
135   signal sample_count : integer range 0 to SAMPLE_PERIOD_CLKS-1 := SAMPLE_PERIOD_CLKS-1;
136

```

```

137 -----
138 -- Sweep scheduler
139 -----
140 type t_sched_state is (
141     SCH_WAIT_PERIOD,
142     SCH_START_FRAME,
143     SCH_WAIT_BUSY_HIGH,
144     SCH_WAIT_BUSY_LOW
145 );
146
147 signal sched_state      : t_sched_state := SCH_WAIT_PERIOD;
148 signal sweep_frame_idx  : integer range 0 to 8 := 0;
149
150 -----
151 -- Sample storage
152 -----
153 signal sample_reg       : std_logic_vector(11 downto 0) := (others => '0');
154 signal sample_addr_reg  : std_logic_vector(9 downto 0)  := (others => '0');
155 signal sample_pending   : std_logic := '0';
156
157 -----
158 -- Simple bus-write FSM
159 -----
160 type t_state is (
161     ST_IDLE,
162     ST_REQ_WRITE,
163     ST_WAIT_ACK
164 );
165
166 signal state : t_state := ST_IDLE;
167
168 -----
169 -- Constants
170 -----
171 constant C_ADC_BASE_ADDR : unsigned(9 downto 0) :=
172     to_unsigned(ADC_BASE_ADDR, 10);
173
174 begin
175
176     bus_req <= bus_req_r;
177
178 -----
179 -- AD7928 SPI master
180 -----
181 u_adc : entity work.ad7928_spi_master
182     generic map (
183         CLK_DIV => ADC_CLK_DIV
184     )
185     port map (
186         clk          => clk_sys,
187         reset_i      => reset_i,
188
189         start_i       => adc_start,
190         channel_i     => adc_next_chan,
191         range_i       => '0',    -- 0..2*REFIN
192         coding_i      => '1',    -- straight binary
193
194         busy_o        => adc_busy,
195         data_valid_o  => adc_data_valid,

```

```

196     primed_o      => adc_primed,
197
198     result_o       => adc_result,
199     result_chan_o  => adc_result_chan,
200     raw_word_o     => adc_raw_word,
201
202     adc_cs_n_o     => adc_cs_n_o,
203     adc_sclk_o     => adc_sclk_o,
204     adc_din_o      => adc_din_o,
205     adc_dout_i     => adc_dout_i
206 );
207
208 -----
209 -- ADC sweep scheduler
210 -- Wait for the programmed sweep period boundary, then issue one complete burst of
211 -- channel requests: 0,1,2,3,4,5,6,7 and one final flush frame requesting 0.
212 -- Because the AD7928 result corresponds to the previous frame, the first
213 -- returned sample of each burst is discarded and the next 8 valid samples
214 -- refresh SPRAM addresses ADC_BASE_ADDR + 0 through ADC_BASE_ADDR + 7.
215 -----
216 p_sample_timer : process(clk_sys)
217     variable v_next_cmd : integer range 0 to 7;
218 begin
219     if rising_edge(clk_sys) then
220         adc_start      <= '0';
221         sweep_start_pulse <= '0';
222
223         if reset_i = '1' then
224             sample_count <= SAMPLE_PERIOD_CLKS-1;
225             adc_start    <= '0';
226             adc_next_chan <= (others => '0');
227             sched_state  <= SCH_WAIT_PERIOD;
228             sweep_frame_idx <= 0;
229         else
230             if sample_count < (SAMPLE_PERIOD_CLKS - 1) then
231                 sample_count <= sample_count + 1;
232             end if;
233
234             case sched_state is
235
236                 when SCH_WAIT_PERIOD =>
237                     if (sample_count = SAMPLE_PERIOD_CLKS - 1) and
238                         (adc_busy = '0') and
239                         (sample_pending = '0') and
240                         (state = ST_IDLE) then
241                         sweep_frame_idx <= 0;
242                         adc_next_chan  <= (others => '0');
243                         sched_state    <= SCH_START_FRAME;
244                     end if;
245
246                 when SCH_START_FRAME =>
247                     if (adc_busy = '0') and (sample_pending = '0') then
248                         adc_start <= '1';
249
250                     if sweep_frame_idx = 0 then
251                         sweep_start_pulse <= '1';
252                         sample_count      <= 0;
253                     end if;
254

```

```

255         sched_state <= SCH_WAIT_BUSY_HIGH;
256     end if;
257
258     when SCH_WAIT_BUSY_HIGH =>
259         if adc_busy = '1' then
260             sched_state <= SCH_WAIT_BUSY_LOW;
261         end if;
262
263     when SCH_WAIT_BUSY_LOW =>
264         if adc_busy = '0' then
265             if sweep_frame_idx = 8 then
266                 sched_state <= SCH_WAIT_PERIOD;
267             else
268                 sweep_frame_idx <= sweep_frame_idx + 1;
269
270                 if sweep_frame_idx = 7 then
271                     v_next_cmd := 0;
272                 else
273                     v_next_cmd := sweep_frame_idx + 1;
274                 end if;
275
276                 adc_next_chan <= std_logic_vector(to_unsigned(v_next_cmd, adc_next_chan'
277                     length));
278                 sched_state <= SCH_START_FRAME;
279             end if;
280         end if;
281
282     when others =>
283         sched_state <= SCH_WAIT_PERIOD;
284
285     end case;
286 end if;
287 end process;
288
289 -----
290 -- Capture the most recent valid ADC sample
291 -----
292 p_sample_capture : process(clk_sys)
293 begin
294     if rising_edge(clk_sys) then
295         if reset_i = '1' then
296             sample_reg <= (others => '0');
297             sample_addr_reg <= (others => '0');
298             sample_pending <= '0';
299             discard_next_sample <= '0';
300         else
301             if sweep_start_pulse = '1' then
302                 discard_next_sample <= '1';
303             end if;
304
305             if adc_data_valid = '1' then
306                 if discard_next_sample = '1' then
307                     discard_next_sample <= '0';
308                 else
309                     sample_reg <= adc_result;
310                     sample_addr_reg <= std_logic_vector(
311                         C_ADC_BASE_ADDR + resize(unsigned(adc_result_chan), sample_addr_reg'length)
312                     );

```

```

313         sample_pending <= '1';
314     end if;
315 end if;
316
317     if (state = ST_WAIT_ACK) and (bus_rsp.ack = '1') then
318         sample_pending <= '0';
319     end if;
320 end if;
321 end if;
322 end process;
323
324 -----
325 -- Bus write client FSM
326 -----
327 p_bus_fsm : process(clk_sys)
328 begin
329     if rising_edge(clk_sys) then
330         if reset_i = '1' then
331             state <= ST_IDLE;
332
333             bus_req_r.req <= '0';
334             bus_req_r.we <= '1';
335             bus_req_r.addr <= std_logic_vector(C_ADC_BASE_ADDR);
336             bus_req_r.wdata <= (others => '0');
337
338         else
339             case state is
340
341                 -----
342                 -- Wait for a captured sample to be available
343                 -----
344                 when ST_IDLE =>
345                     bus_req_r.req <= '0';
346                     bus_req_r.we <= '1';
347                     bus_req_r.addr <= sample_addr_reg;
348                     bus_req_r.wdata <= "0000" & sample_reg;
349
350                     if sample_pending = '1' then
351                         state <= ST_REQ_WRITE;
352                     else
353                         state <= ST_IDLE;
354                     end if;
355
356                 -----
357                 -- Assert one write request
358                 -----
359                 when ST_REQ_WRITE =>
360                     bus_req_r.req <= '1';
361                     bus_req_r.we <= '1';
362                     bus_req_r.addr <= sample_addr_reg;
363                     bus_req_r.wdata <= "0000" & sample_reg;
364
365                     state <= ST_WAIT_ACK;
366
367                 -----
368                 -- Hold req until Bus_Master acknowledges the write
369                 -----
370                 when ST_WAIT_ACK =>
371                     bus_req_r.req <= '1';

```

```

372         bus_req_r.we      <= '1';
373         bus_req_r.addr    <= sample_addr_reg;
374         bus_req_r.wdata   <= "0000" & sample_reg;
375
376         if bus_rsp.ack = '1' then
377             bus_req_r.req <= '0';
378             state         <= ST_IDLE;
379         else
380             state <= ST_WAIT_ACK;
381         end if;
382
383         when others =>
384             state <= ST_IDLE;
385
386     end case;
387 end if;
388 end if;
389 end process;
390
391 end architecture;

```

A.9 uart_rs232_fifo_wrap_controller.vhd

```

1  -- =====
2  -- File: uart_rs232_fifo_wrap_controller.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 16 March 2026
6  -- Last Updated: 14 April 2026
7  --
8  -- Description:
9  --   UART protocol controller wrapper built around uart_rs232_fifo_wrap.
10 --
11 --   This module is responsible for:
12 --       1) UART RX/TX buffering through uart_rs232_fifo_wrap
13 --       2) Packet parsing and framing validation
14 --       3) Packet checksum validation
15 --       4) Command execution sequencing
16 --       5) Formatted TX packet generation
17 --
18 --   This module no longer accesses SPRAM directly.
19 --   Instead, it behaves as a client of the dedicated Bus_Master SPRAM wrapper.
20 --
21 --   Bus interface:
22 --       - bus_req  : request access / transaction
23 --       - bus_ack  : transaction complete / data valid
24 --       - bus_we   : 0=read, 1=write
25 --       - bus_addr : SPRAM word address
26 --       - bus_wdata: data for writes
27 --       - bus_rdata: returned data for reads
28 --
29 --   Packet format:
30 --
31 --       Incoming:
32 --           SOF | LEN | PAYLOAD... | CHKSUM
33 --
34 --       Read request payload:

```

```

35 --      OP_ID | COUNT | ADDR_H | ADDR_L
36 --      Example: 7E 04 0F 10 01 00 DF
37 --
38 --      Write request payload:
39 --      OP_ID | COUNT | ADDR_H | ADDR_L | DATAO_H | DATAO_L | ...
40 --      Example: 7E 12 0A 07 01 00 00 01 BF FF 60 00 20 00 40 00 80 00 FF FF F0
41 --
42 --      Outgoing:
43 --      SOF | LEN | PAYLOAD... | CHKSUM
44 --
45 --      Read response payload:
46 --      OA | COUNT | ADDR_H | ADDR_L | DATAO_H | DATAO_L | ...
47 --
48 --      Write response payload:
49 --      none
50 --
51 --      Error response payload:
52 --      00
53 --
54 --      Notes:
55 --      - SOF is 0x7E
56 --      - LEN counts OP_ID through the end of DATA
57 --      - CHKSUM is 0xFF - sum(PAYLOAD bytes)
58 --      - SOF and CHKSUM are not included in LEN
59 --
60 --      Revision History:
61 --
62 --      2026-03-16
63 --      - Initial version.
64 --      - Separated UART packet handling from Bus_Master.
65 --      - Added RX packet FSM, execution FSM, and TX packet FSM.
66 --      - Implemented packet parsing, checksum validation, request queueing,
67 --      execution sequencing, and formatted reply generation.
68 --
69 --      2026-03-18
70 --      - Refactored the Bus_Master client interface to use structured request
71 --      and response records from bus_pkg.vhd.
72 --      - Replaced flat bus_req / bus_ack / bus_we / bus_addr / bus_wdata /
73 --      bus_rdata ports with bus_req : t_bus_req and bus_rsp : t_bus_rsp.
74 --      - No intended functional change to UART packet handling or execution
75 --      state-machine behavior.
76 --
77 --      2026-03-20
78 --      - Added 0x7E start-of-frame delimiter support to both RX and TX packet
79 --      formats.
80 --      - Updated the RX FSM to require and validate SOF before LEN.
81 --      - Updated the TX FSM to prepend SOF to all transmitted packets.
82 --      - Added response packet support for read transactions.
83 --
84 --      2026-04-05
85 --      - Removed legacy debug LED output port and all associated internal LED
86 --      decode logic.
87 --      - No intended functional change to UART packet handling, bus access, or
88 --      TX/RX behavior.
89 --
90 --      2026-04-07
91 --      - Fixed RX parser so 0x0D is no longer treated as an illegal value
92 --      during payload/checksum reception.
93 --

```

```

94 -- 2026-04-13
95 --   - Updated framing to:
96 --       7E | LEN | OP_ID | COUNT | ADDR_H | ADDR_L | DATA... | CHKSUM
97 --   - LEN now excludes checksum and counts OP_ID through DATA only.
98 --   - Read OP_ID is 0x0F and write/read-response OP_ID is 0x0A.
99 --   - Checksum is now computed as 0xFF - sum(payload bytes).
100 --   - Write requests no longer transmit an acknowledge packet.
101 --
102 -- 2026-04-14
103 --   - Added framed UART error response packet support (7E 01 00 FF) for
104 --     packet checksum failure and invalid request conditions.
105 --   - Implemented single-owner handshake for error response pending flag:
106 --     RX FSM asserts/clears the pending flag, EXEC FSM only requests clear.
107 --   - This prevents LabVIEW from hanging on rejected packets while also
108 --     avoiding multi-driver control logic.
109 --
110 -- =====
111 library ieee;
112 use ieee.std_logic_1164.all;
113 use ieee.numeric_std.all;
114
115 library work;
116 use work.bus_pkg.all;
117
118 entity uart_rs232_fifo_wrap_controller is
119     generic (
120         CLK_FREQ_HZ : integer := 24930000;
121         BAUD         : integer := 9600;
122         DATA_BITS   : integer := 8;
123         STOP_BITS    : integer := 1;
124         PARITY        : integer := 0
125     );
126     port (
127         clk_sys : in  std_logic;
128         reset_i : in  std_logic;
129
130         -- Serial pins
131         RX : in  std_logic;
132         TX : out std_logic;
133
134         -- Client bus to Bus_Master
135         bus_req : out t_bus_req;
136         bus_rsp : in  t_bus_rsp
137     );
138 end entity;
139
140 architecture behavioral of uart_rs232_fifo_wrap_controller is
141
142     -----
143     -- Protocol constants
144     -----
145     constant C_SOF      : std_logic_vector(7 downto 0) := x"7E";
146     constant C_CMD_W    : std_logic_vector(7 downto 0) := x"0A";
147     constant C_CMD_R    : std_logic_vector(7 downto 0) := x"0F";
148     constant C_ERR_PAY  : std_logic_vector(7 downto 0) := x"00";
149
150     constant C_MAX_PKT_BYTES : integer := 64;
151     constant C_MAX_BURST_WORDS : integer := (C_MAX_PKT_BYTES - 5) / 2;
152

```

```

153  constant C_LEN_READ_REQ : integer := 4;
154
155  function f_checksum_next(
156      sum_in  : std_logic_vector(7 downto 0);
157      byte_in : std_logic_vector(7 downto 0)
158  ) return std_logic_vector is
159  begin
160      return std_logic_vector(unsigned(sum_in) + unsigned(byte_in));
161  end function;
162
163  function f_checksum_final(
164      sum_in : std_logic_vector(7 downto 0)
165  ) return std_logic_vector is
166  begin
167      return std_logic_vector(to_unsigned(255, 8) - unsigned(sum_in));
168  end function;
169
170  -----
171  -- Byte array type
172  -----
173  type t_byte_array is array (0 to C_MAX_PKT_BYTES-1) of std_logic_vector(7 downto 0);
174
175  -----
176  -- UART FIFO wrapper signals
177  -----
178  signal rxq_data      : std_logic_vector(7 downto 0);
179  signal rxq_empty     : std_logic;
180  signal rxq_full      : std_logic;
181  signal rxq_rd_en     : std_logic;
182
183  signal txq_data_in   : std_logic_vector(7 downto 0);
184  signal txq_wr_en     : std_logic;
185  signal txq_empty     : std_logic;
186  signal txq_full      : std_logic;
187
188  -----
189  -- RX FSM
190  -----
191  type t_rx_state is (
192      RX_IDLE,
193      RX_SOF_POP,
194      RX_SOF_CAP,
195      RX_LEN_POP,
196      RX_LEN_CAP,
197      RX_BYTE_POP,
198      RX_BYTE_CAP,
199      RX_CHKSUM_POP,
200      RX_CHKSUM_CAP,
201      RX_VALIDATE,
202      RX_QUEUE_REQ,
203      RX_CLEAR
204  );
205
206  -----
207  -- EXEC FSM
208  -----
209  type t_ex_state is (
210      EX_IDLE,
211      EX_READ_REQ,

```

```

212     EX_READ_WAIT,
213     EX_WRITE_REQ,
214     EX_WRITE_WAIT,
215     EX_NEXT,
216     EX_PREP_READ_RSP,
217     EX_START_READ_RSP,
218     EX_PREP_ERR_RSP,
219     EX_START_ERR_RSP
220 );
221
222 -----
223 -- TX FSM
224 -----
225 type t_tx_state is (
226     TX_IDLE,
227     TX_PREP,
228     TX_SEND_SOF,
229     TX_SEND_LEN,
230     TX_SEND_PAYLOAD,
231     TX_SEND_CHKSUM
232 );
233
234 -----
235 -- State registers
236 -----
237 signal rx_state : t_rx_state := RX_IDLE;
238 signal ex_state : t_ex_state := EX_IDLE;
239 signal tx_state : t_tx_state := TX_IDLE;
240
241 -----
242 -- Shared RX pop/capture mechanism
243 -----
244 signal pop_pending : std_logic := '0';
245 signal rx_byte      : std_logic_vector(7 downto 0) := (others => '0');
246
247 -----
248 -- RX packet bookkeeping
249 -----
250 signal rx_buf          : t_byte_array;
251 signal rx_expected_len : integer range 0 to C_MAX_PKT_BYTES := 0;
252 signal rx_index        : integer range 0 to C_MAX_PKT_BYTES := 0;
253 signal rx_checksum_rx   : std_logic_vector(7 downto 0) := (others => '0');
254 signal rx_checksum_calc : std_logic_vector(7 downto 0) := (others => '0');
255
256 -----
257 -- Queued request interface
258 -----
259 signal req_valid      : std_logic := '0';
260 signal req_take       : std_logic := '0';
261 signal req_cmd        : std_logic_vector(7 downto 0) := (others => '0');
262 signal req_count      : integer range 0 to C_MAX_BURST_WORDS := 0;
263 signal req_addr       : std_logic_vector(15 downto 0) := (others => '0');
264 signal req_payload_len : integer range 0 to C_MAX_PKT_BYTES := 0;
265 signal req_buf        : t_byte_array;
266
267 -----
268 -- Error response handshake
269 -----
270 signal err_rsp_pending : std_logic := '0';

```

```

271 signal err_rsp_take      : std_logic := '0';
272
273 -----
274 -- Active execution registers
275 -----
276 signal ex_cmd            : std_logic_vector(7 downto 0) := (others => '0');
277 signal ex_count          : integer range 0 to C_MAX_BURST_WORDS := 0;
278 signal ex_base_addr     : std_logic_vector(15 downto 0) := (others => '0');
279 signal ex_index         : integer range 0 to C_MAX_BURST_WORDS := 0;
280 signal ex_curr_addr     : std_logic_vector(9 downto 0) := (others => '0');
281 signal ex_curr_wdata    : std_logic_vector(15 downto 0) := (others => '0');
282 signal ex_buf           : t_byte_array;
283
284 -----
285 -- TX payload interface
286 -----
287 signal tx_payload_buf : t_byte_array;
288 signal tx_payload_len : integer range 0 to C_MAX_PKT_BYTES := 0;
289 signal tx_checksum    : std_logic_vector(7 downto 0) := (others => '0');
290 signal tx_len_byte    : std_logic_vector(7 downto 0) := (others => '0');
291 signal tx_index       : integer range 0 to C_MAX_PKT_BYTES := 0;
292 signal tx_start       : std_logic := '0';
293 signal tx_busy        : std_logic := '0';
294
295 begin
296
297 -----
298 -- UART wrapper instantiation
299 -----
300 u_uart_wrap : entity work. uart_rs232_fifo_wrap
301   generic map (
302     CLK_FREQ_HZ => CLK_FREQ_HZ,
303     BAUD        => BAUD,
304     DATA_BITS  => DATA_BITS,
305     STOP_BITS   => STOP_BITS,
306     PARITY      => PARITY
307   )
308   port map (
309     clk_sys => clk_sys,
310     reset_i => reset_i,
311
312     RX => RX,
313     TX => TX,
314
315     rxq_data  => rxq_data,
316     rxq_empty => rxq_empty,
317     rxq_full  => rxq_full,
318     rxq_rd_en => rxq_rd_en,
319
320     txq_data_in => txq_data_in,
321     txq_wr_en   => txq_wr_en,
322     txq_empty   => txq_empty,
323     txq_full    => txq_full
324   );
325
326 -----
327 -- RX FSM
328 -----
329 p_rx_fsm : process(clk_sys)

```

```

330     variable v_payload_len : integer;
331     variable v_count       : integer;
332     variable v_ok          : boolean;
333 begin
334     if rising_edge(clk_sys) then
335
336         rxq_rd_en <= '0';
337
338         if reset_i = '1' then
339             rx_state      <= RX_IDLE;
340             pop_pending    <= '0';
341             rx_byte        <= (others => '0');
342             rx_expected_len <= 0;
343             rx_index       <= 0;
344             rx_checksum_rx  <= (others => '0');
345             rx_checksum_calc <= (others => '0');
346
347             req_valid      <= '0';
348             req_cmd         <= (others => '0');
349             req_count       <= 0;
350             req_addr        <= (others => '0');
351             req_payload_len <= 0;
352
353             err_rsp_pending <= '0';
354
355             for i in 0 to C_MAX_PKT_BYTES-1 loop
356                 rx_buf(i) <= (others => '0');
357                 req_buf(i) <= (others => '0');
358             end loop;
359
360         else
361
362             if pop_pending = '1' then
363                 rx_byte      <= rxq_data;
364                 pop_pending <= '0';
365             end if;
366
367             if req_take = '1' then
368                 req_valid <= '0';
369             end if;
370
371             if err_rsp_take = '1' then
372                 err_rsp_pending <= '0';
373             end if;
374
375             case rx_state is
376
377                 when RX_IDLE =>
378                     if rxq_empty = '0' then
379                         rx_state <= RX_SOF_POP;
380                     end if;
381
382                 when RX_SOF_POP =>
383                     if rxq_empty = '0' then
384                         rxq_rd_en <= '1';
385                         pop_pending <= '1';
386                         rx_state <= RX_SOF_CAP;
387                     end if;
388

```

```

389 when RX_SOF_CAP =>
390     if pop_pending = '0' then
391         if rx_byte = C_SOF then
392             rx_state <= RX_LEN_POP;
393         else
394             rx_state <= RX_IDLE;
395         end if;
396     end if;
397
398 when RX_LEN_POP =>
399     if rxq_empty = '0' then
400         rxq_rd_en <= '1';
401         pop_pending <= '1';
402         rx_state <= RX_LEN_CAP;
403     end if;
404
405 when RX_LEN_CAP =>
406     if pop_pending = '0' then
407         rx_expected_len <= to_integer(unsigned(rx_byte));
408         rx_index <= 0;
409         rx_checksum_calc <= (others => '0');
410         rx_checksum_rx <= (others => '0');
411
412         if (to_integer(unsigned(rx_byte)) >= 2) and
413             (to_integer(unsigned(rx_byte)) <= C_MAX_PKT_BYTES) then
414             rx_state <= RX_BYTE_POP;
415         else
416             err_rsp_pending <= '1';
417             rx_state <= RX_CLEAR;
418         end if;
419     end if;
420
421 when RX_BYTE_POP =>
422     if rx_index < rx_expected_len then
423         if rxq_empty = '0' then
424             rxq_rd_en <= '1';
425             pop_pending <= '1';
426             rx_state <= RX_BYTE_CAP;
427         end if;
428     else
429         rx_state <= RX_CHKSUM_POP;
430     end if;
431
432 when RX_BYTE_CAP =>
433     if pop_pending = '0' then
434         rx_buf(rx_index) <= rx_byte;
435         rx_checksum_calc <= f_checksum_next(rx_checksum_calc, rx_byte);
436
437         if (rx_index + 1) < rx_expected_len then
438             rx_index <= rx_index + 1;
439             rx_state <= RX_BYTE_POP;
440         else
441             rx_index <= rx_index + 1;
442             rx_state <= RX_CHKSUM_POP;
443         end if;
444     end if;
445
446 when RX_CHKSUM_POP =>
447     if rxq_empty = '0' then

```

```

448         rxq_rd_en    <= '1';
449         pop_pending  <= '1';
450         rx_state     <= RX_CHKSUM_CAP;
451     end if;
452
453     when RX_CHKSUM_CAP =>
454         if pop_pending = '0' then
455             rx_checksum_rx <= rx_byte;
456             rx_state      <= RX_VALIDATE;
457         end if;
458
459     when RX_VALIDATE =>
460         if rx_checksum_rx = f_checksum_final(rx_checksum_calc) then
461             rx_state <= RX_QUEUE_REQ;
462         else
463             err_rsp_pending <= '1';
464             rx_state      <= RX_CLEAR;
465         end if;
466
467     when RX_QUEUE_REQ =>
468         if req_valid = '0' then
469
470             v_payload_len := rx_expected_len;
471             v_count       := 0;
472             v_ok          := false;
473
474             if rx_buf(0) = C_CMD_R then
475                 if rx_expected_len = C_LEN_READ_REQ then
476                     v_count := to_integer(unsigned(rx_buf(1)));
477                     if (v_count >= 1) and
478                         (v_count <= C_MAX_BURST_WORDS) and
479                         ((2 * v_count) <= C_MAX_PKT_BYTES) then
480                         v_ok := true;
481                     end if;
482                 end if;
483
484                 elsif rx_buf(0) = C_CMD_W then
485                     if v_payload_len >= 4 then
486                         v_count := to_integer(unsigned(rx_buf(1)));
487                         if (v_count >= 1) and
488                             (v_count <= C_MAX_BURST_WORDS) and
489                             (v_payload_len = (4 + 2*v_count)) then
490                             v_ok := true;
491                         end if;
492                     end if;
493                 end if;
494
495                 if v_ok then
496                     req_cmd      <= rx_buf(0);
497                     req_count    <= v_count;
498                     req_addr     <= rx_buf(2) & rx_buf(3);
499                     req_payload_len <= v_payload_len;
500                     req_valid    <= '1';
501
502                     for i in 0 to C_MAX_PKT_BYTES-1 loop
503                         req_buf(i) <= rx_buf(i);
504                     end loop;
505
506                     rx_state <= RX_CLEAR;

```

```

507         else
508             err_rsp_pending <= '1';
509             rx_state        <= RX_CLEAR;
510         end if;
511     end if;
512
513     when RX_CLEAR =>
514         rx_expected_len <= 0;
515         rx_index        <= 0;
516         rx_checksum_rx  <= (others => '0');
517         rx_checksum_calc <= (others => '0');
518         pop_pending     <= '0';
519         rx_byte         <= (others => '0');
520
521         for i in 0 to C_MAX_PKT_BYTES-1 loop
522             rx_buf(i) <= (others => '0');
523         end loop;
524
525         rx_state <= RX_IDLE;
526
527         when others =>
528             rx_state <= RX_IDLE;
529         end case;
530     end if;
531 end if;
532 end process;
533
534 -----
535 -- EXEC FSM
536 -----
537 p_exec_fsm : process(clk_sys)
538 begin
539     if rising_edge(clk_sys) then
540
541         bus_req.req    <= '0';
542         bus_req.we     <= '0';
543         bus_req.addr   <= (others => '0');
544         bus_req.wdata  <= (others => '0');
545
546         tx_start      <= '0';
547         req_take       <= '0';
548         err_rsp_take  <= '0';
549
550         if reset_i = '1' then
551             ex_state    <= EX_IDLE;
552             ex_cmd      <= (others => '0');
553             ex_count    <= 0;
554             ex_base_addr <= (others => '0');
555             ex_index    <= 0;
556             ex_curr_addr <= (others => '0');
557             ex_curr_wdata <= (others => '0');
558             tx_payload_len <= 0;
559
560             for i in 0 to C_MAX_PKT_BYTES-1 loop
561                 ex_buf(i) <= (others => '0');
562                 tx_payload_buf(i) <= (others => '0');
563             end loop;
564
565         else

```

```

566     case ex_state is
567
568     when EX_IDLE =>
569         ex_index <= 0;
570
571         if (err_rsp_pending = '1') and (tx_busy = '0') then
572             ex_state <= EX_PREP_ERR_RSP;
573
574         elsif (req_valid = '1') and (tx_busy = '0') then
575             ex_cmd      <= req_cmd;
576             ex_count    <= req_count;
577             ex_base_addr <= req_addr;
578             ex_index    <= 0;
579
580             for i in 0 to C_MAX_PKT_BYTES-1 loop
581                 ex_buf(i) <= req_buf(i);
582             end loop;
583
584             req_take <= '1';
585
586             if req_cmd = C_CMD_R then
587                 ex_state <= EX_READ_REQ;
588             else
589                 ex_state <= EX_WRITE_REQ;
590             end if;
591         end if;
592
593     when EX_READ_REQ =>
594         ex_curr_addr <= std_logic_vector(unsigned(ex_base_addr(9 downto 0)) +
595             to_unsigned(ex_index, 10));
596
597         bus_req.req    <= '1';
598         bus_req.we     <= '0';
599         bus_req.addr   <= std_logic_vector(unsigned(ex_base_addr(9 downto 0)) +
600             to_unsigned(ex_index, 10));
601
602         if bus_rsp.ack = '1' then
603             ex_state <= EX_READ_WAIT;
604         else
605             ex_state <= EX_READ_REQ;
606         end if;
607
608     when EX_READ_WAIT =>
609         tx_payload_buf(4 + 2*ex_index) <= bus_rsp.rdata(15 downto 8);
610         tx_payload_buf(5 + 2*ex_index) <= bus_rsp.rdata(7  downto 0);
611         ex_state <= EX_NEXT;
612
613     when EX_WRITE_REQ =>
614         ex_curr_addr <= std_logic_vector(unsigned(ex_base_addr(9 downto 0)) +
615             to_unsigned(ex_index, 10));
616         ex_curr_wdata <= ex_buf(4 + 2*ex_index) & ex_buf(5 + 2*ex_index);
617
618         bus_req.req    <= '1';
619         bus_req.we     <= '1';
620         bus_req.addr   <= std_logic_vector(unsigned(ex_base_addr(9 downto 0)) +
621             to_unsigned(ex_index, 10));
622         bus_req.wdata <= ex_buf(4 + 2*ex_index) & ex_buf(5 + 2*ex_index);
623
624         if bus_rsp.ack = '1' then

```

```

621         ex_state <= EX_NEXT;
622     else
623         ex_state <= EX_WRITE_REQ;
624     end if;
625
626 when EX_WRITE_WAIT =>
627     ex_state <= EX_NEXT;
628
629 when EX_NEXT =>
630     if (ex_index + 1) < ex_count then
631         ex_index <= ex_index + 1;
632
633         if ex_cmd = C_CMD_R then
634             ex_state <= EX_READ_REQ;
635         else
636             ex_state <= EX_WRITE_REQ;
637         end if;
638     else
639         if ex_cmd = C_CMD_R then
640             ex_state <= EX_PREP_READ_RSP;
641         else
642             ex_state <= EX_IDLE;
643         end if;
644     end if;
645
646 when EX_PREP_READ_RSP =>
647     if tx_busy = '0' then
648         tx_payload_buf(0) <= C_CMD_W;
649         tx_payload_buf(1) <= std_logic_vector(to_unsigned(ex_count, 8));
650         tx_payload_buf(2) <= ex_base_addr(15 downto 8);
651         tx_payload_buf(3) <= ex_base_addr(7 downto 0);
652         tx_payload_len <= 4 + 2 * ex_count;
653         ex_state <= EX_START_READ_RSP;
654     end if;
655
656 when EX_START_READ_RSP =>
657     if tx_busy = '0' then
658         tx_start <= '1';
659         ex_state <= EX_IDLE;
660     end if;
661
662 when EX_PREP_ERR_RSP =>
663     if tx_busy = '0' then
664         tx_payload_buf(0) <= C_ERR_PAY;
665         tx_payload_len <= 1;
666         ex_state <= EX_START_ERR_RSP;
667     end if;
668
669 when EX_START_ERR_RSP =>
670     if tx_busy = '0' then
671         tx_start <= '1';
672         err_rsp_take <= '1';
673         ex_state <= EX_IDLE;
674     end if;
675
676 when others =>
677     ex_state <= EX_IDLE;
678 end case;
679 end if;

```

```

680     end if;
681 end process;
682
683 -----
684 -- TX FSM
685 -----
686 p_tx_fsm : process(clk_sys)
687 begin
688     if rising_edge(clk_sys) then
689
690         txq_wr_en <= '0';
691
692         if reset_i = '1' then
693             tx_state    <= TX_IDLE;
694             tx_busy     <= '0';
695             tx_index    <= 0;
696             tx_checksum <= (others => '0');
697             tx_len_byte <= (others => '0');
698             txq_data_in <= (others => '0');
699
700         else
701             case tx_state is
702
703                 when TX_IDLE =>
704                     tx_busy <= '0';
705                     tx_index <= 0;
706
707                     if tx_start = '1' then
708                         tx_busy <= '1';
709                         tx_state <= TX_PREP;
710                     end if;
711
712                 when TX_PREP =>
713                     tx_busy <= '1';
714                     tx_len_byte <= std_logic_vector(to_unsigned(tx_payload_len, 8));
715                     tx_checksum <= (others => '0');
716                     tx_index <= 0;
717                     tx_state <= TX_SEND_SOF;
718
719                 when TX_SEND_SOF =>
720                     tx_busy <= '1';
721                     if txq_full = '0' then
722                         txq_data_in <= C_SOF;
723                         txq_wr_en <= '1';
724                         tx_state <= TX_SEND_LEN;
725                     end if;
726
727                 when TX_SEND_LEN =>
728                     tx_busy <= '1';
729                     if txq_full = '0' then
730                         txq_data_in <= tx_len_byte;
731                         txq_wr_en <= '1';
732                         tx_index <= 0;
733                         tx_state <= TX_SEND_PAYLOAD;
734                     end if;
735
736                 when TX_SEND_PAYLOAD =>
737                     tx_busy <= '1';
738

```

```

739         if tx_index < tx_payload_len then
740             if txq_full = '0' then
741                 txq_data_in <= tx_payload_buf(tx_index);
742                 txq_wr_en   <= '1';
743                 tx_checksum <= f_checksum_next(tx_checksum, tx_payload_buf(tx_index));
744                 tx_index    <= tx_index + 1;
745             end if;
746         else
747             tx_state <= TX_SEND_CHKSUM;
748         end if;
749
750         when TX_SEND_CHKSUM =>
751             tx_busy <= '1';
752             if txq_full = '0' then
753                 txq_data_in <= f_checksum_final(tx_checksum);
754                 txq_wr_en   <= '1';
755                 tx_state    <= TX_IDLE;
756             end if;
757
758             when others =>
759                 tx_state <= TX_IDLE;
760             end case;
761         end if;
762     end if;
763 end process;
764
765 end architecture;

```

A.10 UART_rs232_fifo_wrap.vhd

```

1  -- =====
2  -- File: uart_rs232_fifo_wrap.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 04 March 2026
6  --
7  -- Description:
8  --   Wrapper around uart_rs232 that adds simple RX and TX FIFOs.
9  --   This module decouples the UART core from downstream logic by:
10 --   - buffering received bytes (RX FIFO)
11 --   - buffering bytes to transmit (TX FIFO)
12 --
13 --   RX Operation:
14 --   - Accepts UART RX bytes when RX FIFO is not full.
15 --   - Presents FIFO output as rxq_data.
16 --   - User logic pops bytes with a 1-cycle pulse on rxq_rd_en.
17 --
18 --   TX Operation:
19 --   - User logic pushes bytes with a 1-cycle pulse on txq_wr_en.
20 --   - When TX FIFO is non-empty, uart_tx_valid is asserted continuously.
21 --   - FIFO is popped only when the UART asserts tx_ready (TX idle),
22 --     ensuring bytes transmit back-to-back as fast as the UART allows.
23 --
24 -- Interfaces:
25 --   - Connects directly to: uart_rs232.vhd
26 --   - FIFO implementation: My_Own_FN_FIFO.vhd
27 --

```

```

28 -- Notes:
29 --   - Single-clock domain: clk_sys
30 --   - Active-high synchronous reset: reset_i
31 --   - FIFO push/pop strobes are assumed to be single-clock pulses.
32 --
33 -- Revision History:
34 --   2026-03-04 - Initial version.
35 -- =====
36
37 library ieee;
38 use ieee.std_logic_1164.all;
39 use ieee.numeric_std.all;
40
41 entity uart_rs232_fifo_wrap is
42   generic (
43     CLK_FREQ_HZ : integer := 24930000;
44     BAUD         : integer := 9600;
45     DATA_BITS   : integer := 8;
46     STOP_BITS    : integer := 1;
47     PARITY       : integer := 0
48   );
49   port (
50     clk_sys : in  std_logic;
51     reset_i : in  std_logic;  -- active-high synchronous reset
52
53     -- Serial pins
54     RX : in  std_logic;
55     TX : out std_logic;
56
57     -- =====
58     -- RX FIFO (bytes received)
59     -- =====
60     rxq_data  : out std_logic_vector(7 downto 0);  -- current FIFO output
61     rxq_empty : out std_logic;
62     rxq_full  : out std_logic;
63     rxq_rd_en : in  std_logic;                    -- pulse 1 clk to pop
64
65     -- =====
66     -- TX FIFO (bytes to send)
67     -- =====
68     txq_data_in : in  std_logic_vector(7 downto 0); -- byte to enqueue for TX
69     txq_wr_en   : in  std_logic;                    -- pulse 1 clk to push
70     txq_empty   : out std_logic;
71     txq_full    : out std_logic
72   );
73 end entity;
74
75 architecture rtl of uart_rs232_fifo_wrap is
76
77   -----
78   -- FIFO depth
79   -- Keep these comfortably larger than a single command/response packet so
80   -- short host bursts do not overflow the software-facing queues.
81   -----
82   constant C_RX_FIFO_DEPTH : integer := 256;
83   constant C_TX_FIFO_DEPTH : integer := 256;
84
85   -----
86   -- UART signals

```

```

87  -----
88  signal uart_rx_data      : std_logic_vector(7 downto 0);
89  signal uart_rx_valid    : std_logic;
90  signal uart_rx_ready    : std_logic;
91  signal uart_rx_framing_err : std_logic;
92  signal uart_rx_parity_err : std_logic;
93  signal uart_rx_overrun   : std_logic;
94
95  signal uart_tx_data      : std_logic_vector(7 downto 0);
96  signal uart_tx_valid    : std_logic;
97  signal uart_tx_ready    : std_logic;
98
99  -----
100  -- RX FIFO signals (your FIFO)
101  -----
102  signal rx_fifo_q        : std_logic_vector(7 downto 0);
103  signal rx_fifo_empty    : std_logic;
104  signal rx_fifo_full     : std_logic;
105  signal rx_fifo_push     : std_logic;
106
107  -----
108  -- TX FIFO signals (your FIFO)
109  -----
110  signal tx_fifo_q        : std_logic_vector(7 downto 0);
111  signal tx_fifo_empty    : std_logic;
112  signal tx_fifo_full     : std_logic;
113  signal tx_fifo_pop      : std_logic;
114
115  begin
116
117  -----
118  -- Export FIFO status/data to top level
119  -----
120  rxq_data  <= rx_fifo_q;
121  rxq_empty <= rx_fifo_empty;
122  rxq_full  <= rx_fifo_full;
123
124  txq_empty <= tx_fifo_empty;
125  txq_full  <= tx_fifo_full;
126
127  -----
128  -- RX path: UART -> RX FIFO
129  -- Only accept UART byte when FIFO not full.
130  -----
131  uart_rx_ready <= not rx_fifo_full;
132
133  -- Push into RX FIFO when UART asserts rx_valid and we are ready
134  rx_fifo_push <= uart_rx_valid and uart_rx_ready;
135
136  -----
137  -- TX path: TX FIFO -> UART (send ASAP)
138  -- Present FIFO Q as uart_tx_data.
139  -- Assert uart_tx_valid whenever TX FIFO not empty.
140  -- Pop FIFO only when UART is ready to accept (handshake).
141  -----
142  uart_tx_data <= tx_fifo_q;
143  uart_tx_valid <= not tx_fifo_empty;
144
145  tx_fifo_pop <= uart_tx_ready and (not tx_fifo_empty);

```

```

146
147 -----
148 -- Instantiate UART
149 -----
150 u_uart : entity work. uart_rs232
151     generic map (
152         CLK_FREQ_HZ => CLK_FREQ_HZ,
153         BAUD         => BAUD,
154         DATA_BITS   => DATA_BITS,
155         STOP_BITS    => STOP_BITS,
156         PARITY       => PARITY
157     )
158     port map (
159         clk    => clk_sys,
160         reset  => reset_i,
161
162         rx     => RX,
163         tx     => TX,
164
165         rx_data      => uart_rx_data,
166         rx_valid     => uart_rx_valid,
167         rx_ready     => uart_rx_ready,
168         rx_framing_err => uart_rx_framing_err,
169         rx_parity_err => uart_rx_parity_err,
170         rx_overrun   => uart_rx_overrun,
171
172         tx_data  => uart_tx_data,
173         tx_valid => uart_tx_valid,
174         tx_ready => uart_tx_ready
175     );
176
177 -----
178 -- Instantiate RX FIFO (your FIFO)
179 -----
180 u_rx_fifo : entity work.My_Own_FN_FIFO
181     generic map (
182         WIDTH => 8,
183         DEPTH => C_RX_FIFO_DEPTH
184     )
185     port map (
186         clk    => clk_sys,
187         rst    => reset_i,
188
189         push   => rx_fifo_push,
190         din    => uart_rx_data,
191
192         pop    => rxq_rd_en,
193         dout   => rx_fifo_q,
194
195         empty  => rx_fifo_empty,
196         full   => rx_fifo_full
197     );
198
199 -----
200 -- Instantiate TX FIFO (your FIFO)
201 -----
202 u_tx_fifo : entity work.My_Own_FN_FIFO
203     generic map (
204         WIDTH => 8,

```

```

205     DEPTH => C_TX_FIFO_DEPTH
206 )
207 port map (
208     clk    => clk_sys,
209     rst    => reset_i,
210
211     push   => txq_wr_en,
212     din    => txq_data_in,
213
214     pop    => tx_fifo_pop,
215     dout   => tx_fifo_q,
216
217     empty  => tx_fifo_empty,
218     full   => tx_fifo_full
219 );
220
221 end architecture;

```

A.11 UART_rs232.vhd

```

1  -- =====
2  -- Module      : uart_rs232
3  -- Author      : Kevin Lyon
4  -- Description : UART (RS-232 style) transmitter and
5  --              receiver with configurable baud rate,
6  --              data width, stop bits, and parity.
7  --
8  -- Features:
9  --   - Parameterizable baud rate via CLK_FREQ_HZ and BAUD generics
10 --   - Configurable DATA_BITS (typically 5-8)
11 --   - Configurable STOP_BITS (1 or 2)
12 --   - Optional parity:
13 --       0 = None
14 --       1 = Even
15 --       2 = Odd
16 --   - Independent TX and RX state machines
17 --   - Single-cycle handshake interface
18 --   - Framing, parity, and overrun error detection
19 --
20 -- Clocking:
21 --   - Single clock domain
22 --   - Synchronous active-high reset
23 --   - Baud timing derived from integer clock division
24 --
25 -- TX Interface:
26 --   tx_data    : Data to transmit
27 --   tx_valid   : Assert to request transmission
28 --   tx_ready   : High when transmitter is idle
29 --
30 -- RX Interface:
31 --   rx_data    : Received data word
32 --   rx_valid   : Asserted when data is available
33 --   rx_ready   : Assert to acknowledge data
34 --   rx_framing_err : Stop bit error detected
35 --   rx_parity_err  : Parity error detected
36 --   rx_overrun   : New data arrived before previous was read
37 --

```

```

38 -- Revision History:
39 --   2-27-2026 - Initial version
40 --   2-28-2026 - Dropped shift register primitive in favor of indexed
41 --               register for TX/RX
42 --   4-14-2026 - Added 2-flop synchronizer on asynchronous RX input.
43 --               - Reworked RX timing to use 16x oversampling.
44 --               - Intended to reduce intermittent UART packet parse
45 --               failures caused by sampling-position sensitivity.
46 --
47 --   **I need to add RX and TX buffer registers**
48 --   **But they are not needed for this homework**
49 --   **RX and TX FIFO added in wrapper**
50 -- =====
51 library ieee;
52 use ieee.std_logic_1164.all;
53 use ieee.numeric_std.all;
54
55 entity uart_rs232 is
56   generic (
57     CLK_FREQ_HZ : integer := 24930000;
58     BAUD         : integer := 9600;
59
60     DATA_BITS   : integer := 8;
61     STOP_BITS    : integer := 1;
62
63     -- PARITY: 0=None, 1=Even, 2=Odd
64     PARITY       : integer := 0
65   );
66   port (
67     clk    : in  std_logic;
68     reset  : in  std_logic; -- active-high synchronous reset
69
70     -- Serial pins
71     rx     : in  std_logic;
72     tx     : out std_logic;
73
74     -- Receive interface
75     rx_data      : out std_logic_vector(7 downto 0);
76     rx_valid     : out std_logic;
77     rx_ready     : in  std_logic;
78     rx_framing_err : out std_logic;
79     rx_parity_err : out std_logic;
80     rx_overrun   : out std_logic;
81
82     -- Transmit interface
83     tx_data      : in  std_logic_vector(7 downto 0);
84     tx_valid     : in  std_logic;
85     tx_ready     : out std_logic
86   );
87 end entity;
88
89 architecture behavioral of uart_rs232 is
90
91   constant BAUD_DIV : integer := (CLK_FREQ_HZ / BAUD);
92   constant HALF_DIV : integer := (CLK_FREQ_HZ / BAUD) / 2;
93   constant RX_OVERSAMPLE : integer := 16;
94   constant RX_TICK_DIV   : integer := CLK_FREQ_HZ / (BAUD * RX_OVERSAMPLE);
95   constant RX_HALF_BIT_TICKS : integer := RX_OVERSAMPLE / 2;
96

```

```

97  function parity_xor(d : std_logic_vector(7 downto 0); nbits : integer) return std_logic is
98      variable p : std_logic := '0';
99  begin
100      for i in 0 to nbits-1 loop
101          p := p xor d(i);
102      end loop;
103      return p;
104  end function;
105
106  -- =====
107  -- TX
108  -- =====
109  type t_tx_state is (
110      TX_IDLE,
111      TX_START,
112      TX_SEND,
113      TX_PAR,
114      TX_STOP
115  );
116  signal tx_state      : t_tx_state := TX_IDLE;
117  signal tx_bit_timer  : integer range 0 to BAUD_DIV := 0;
118  signal tx_bit_idx    : integer range 0 to 7 := 0;
119
120  signal tx_shift      : std_logic_vector(7 downto 0) := (others => '0');
121  signal tx_par_bit    : std_logic := '0';
122  signal tx_stop_cnt   : integer range 0 to 2 := 0;
123
124  signal tx_line       : std_logic := '1';
125
126  -- =====
127  -- RX
128  -- =====
129  type t_rx_state is (
130      RX_IDLE,
131      RX_START,
132      RX_RECV,
133      RX_PAR,
134      RX_STOP
135  );
136  signal rx_state      : t_rx_state := RX_IDLE;
137  signal rx_bit_timer  : integer range 0 to RX_OVERSAMPLE := 0;
138  signal rx_bit_idx    : integer range 0 to 7 := 0;
139  signal rx_tick_cnt   : integer range 0 to RX_TICK_DIV := 0;
140
141  signal rx_shift      : std_logic_vector(7 downto 0) := (others => '0');
142  signal rx_par_bit    : std_logic := '0';
143  signal rx_stop_cnt   : integer range 0 to 2 := 0;
144
145  signal rx_meta       : std_logic := '1';
146  signal rx_sync       : std_logic := '1';
147  signal rx_sync_prev  : std_logic := '1';
148
149  signal rx_valid_r     : std_logic := '0';
150  signal rx_framing_err_r : std_logic := '0';
151  signal rx_parity_err_r : std_logic := '0';
152  signal rx_overrun_r   : std_logic := '0';
153
154  begin
155      tx <= tx_line;

```

```

156
157 rx_data      <= rx_shift;
158 rx_valid     <= rx_valid_r;
159 rx_framing_err <= rx_framing_err_r;
160 rx_parity_err <= rx_parity_err_r;
161 rx_overrun   <= rx_overrun_r;
162
163 tx_ready <= '1' when tx_state = TX_IDLE else '0'; --Signal ready to recieve when idle
164
165 -- =====
166 -- TX process
167 -- =====
168 p_tx : process(clk)
169 begin
170     if rising_edge(clk) then
171         if reset = '1' then --Let's deal with reset first
172             tx_state      <= TX_IDLE;
173             tx_bit_timer <= 0;
174             tx_bit_idx    <= 0;
175             tx_shift      <= (others => '0');
176             tx_par_bit    <= '0';
177             tx_stop_cnt   <= 0;
178             tx_line       <= '1';
179         else
180             case tx_state is --Now the meat of our TX state machine
181                 when TX_IDLE =>
182                     tx_line <= '1';
183                     tx_bit_timer <= 0;
184                     tx_bit_idx  <= 0;
185                     tx_stop_cnt <= 0;
186
187                     if tx_valid = '1' then --Data on tx_data is ready to go
188                         tx_shift <= tx_data; --Load the outgoing shift register
189
190                         if PARITY = 0 then
191                             tx_par_bit <= '0';
192                         elsif PARITY = 1 then -- even
193                             tx_par_bit <= parity_xor(tx_data, DATA_BITS); --Calculate parity bit
194                         else -- odd
195                             tx_par_bit <= not parity_xor(tx_data, DATA_BITS); --Calculate negated
196                                     parity bit
197                         end if;
198
199                         tx_state <= TX_START; --Locked and Loaded...let's light this candle
200                         tx_line  <= '0'; --Drop TX to signal start condition
201                         tx_bit_timer <= BAUD_DIV - 1;
202                     end if;
203
204                     when TX_START =>
205                         if tx_bit_timer = 0 then --Start bit complete
206                             tx_state <= TX_SEND; --Next advance state machine to send data
207                             tx_line  <= tx_shift(0); --Present LSB to TX
208                             tx_bit_idx <= 0; --Reset our data bit counter index
209                             tx_bit_timer <= BAUD_DIV - 1; --Reset the timer for first bit
210                         else
211                             tx_bit_timer <= tx_bit_timer - 1; --Count down to 0
212                         end if;
213
214                     when TX_SEND =>

```

```

214     if tx_bit_timer = 0 then
215         if tx_bit_idx = DATA_BITS - 1 then    --Time to send the stop bit
216             if PARITY = 0 then                --For no Parity
217                 tx_state <= TX_STOP;          --Next advance state machine to send stop
                bits
218                 tx_line <= '1';              --Raise TX line back up...I always forget this
219                 tx_stop_cnt <= STOP_BITS;
220                 tx_bit_timer <= BAUD_DIV - 1;  --Reset the timer for stop bit
221             else
222                 tx_state <= TX_PAR;           --If Even or Odd Par
223                 tx_line <= tx_par_bit;        --Present Bit on TX
224                 tx_bit_timer <= BAUD_DIV - 1;  --Reset the bit timer
225             end if;
226         else
227             tx_bit_idx <= tx_bit_idx + 1;      --If we have more data bits left to send
228             ...
229             tx_line <= tx_shift(tx_bit_idx + 1); --Send next byte **Note we are not
                shifting
230             --any more as I want to preserve this data for
231             --Parity Calculation so "shift" is now just an
232             --indexed reg. I should rename it...
233             tx_bit_timer <= BAUD_DIV - 1;      --Reset the bit timer
234         end if;
235     else
236         tx_bit_timer <= tx_bit_timer - 1;      --Count down to 0
237     end if;
238
239     when TX_PAR =>                            --Let's send that parity bit
240         if tx_bit_timer = 0 then
241             tx_state <= TX_STOP;              --Next clock will advance state machine to stop
242             bit transmission
243             tx_line <= '1';
244             tx_stop_cnt <= STOP_BITS;          --Let's go ahead and set the number of stop
245             bits
246             tx_bit_timer <= BAUD_DIV - 1;      --Reset the bit timer
247         else
248             tx_bit_timer <= tx_bit_timer - 1;  --Count down to 0
249         end if;
250
251     when TX_STOP =>                            --Send stop bits
252         if tx_bit_timer = 0 then
253             if tx_stop_cnt <= 1 then
254                 tx_state <= TX_IDLE;          --Back home we go...
255                 tx_line <= '1';
256             else
257                 tx_stop_cnt <= tx_stop_cnt - 1;
258                 tx_line <= '1';
259                 tx_bit_timer <= BAUD_DIV - 1;
260             end if;
261         else
262             tx_bit_timer <= tx_bit_timer - 1;
263         end if;
264     end case;
265 end if;
266 end process;
267
268 -- =====
269 -- RX process

```

```

268 -- =====
269 p_rx : process(clk)                                --Entirely parallel recieve circuit
270     variable pcalc : std_logic;                    --Variable to accumulate parity
271     variable rx_tick : std_logic;
272 begin
273     if rising_edge(clk) then
274         rx_tick := '0';
275
276         if reset = '1' then                        --Again let's deal with reset first
277             rx_state      <= RX_IDLE;
278             rx_bit_timer  <= 0;
279             rx_bit_idx    <= 0;                    --We need an idex anyways so
280             rx_tick_cnt  <= 0;
281             rx_shift     <= (others => '0');    --no longer using the shift primitive
282             rx_par_bit   <= '0';
283             rx_stop_cnt  <= 0;
284             rx_meta      <= '1';
285             rx_sync      <= '1';
286             rx_sync_prev <= '1';
287
288             rx_valid_r   <= '0';
289             rx_framing_err_r <= '0';
290             rx_parity_err_r <= '0';
291             rx_overrun_r <= '0';
292
293         else
294             rx_meta      <= rx;
295             rx_sync_prev <= rx_sync;
296             rx_sync      <= rx_meta;
297
298             if rx_tick_cnt = RX_TICK_DIV - 1 then
299                 rx_tick_cnt <= 0;
300                 rx_tick := '1';
301             else
302                 rx_tick_cnt <= rx_tick_cnt + 1;
303             end if;
304
305             -- hold valid until consumed
306             if rx_valid_r = '1' and rx_ready = '1' then
307                 rx_valid_r <= '0';
308                 rx_framing_err_r <= '0';
309                 rx_parity_err_r <= '0';
310             end if;
311
312             case rx_state is
313                 when RX_IDLE =>
314                     rx_bit_timer <= 0;
315                     rx_bit_idx  <= 0;
316                     rx_stop_cnt <= 0;
317
318                     if (rx_sync_prev = '1') and (rx_sync = '0') then
319                         rx_state      <= RX_START;
320                         rx_bit_timer <= RX_HALF_BIT_TICKS;
321                     end if;
322
323                 when RX_START =>
324                     if rx_tick = '1' then
325                         if rx_bit_timer = 0 then
326                             if rx_sync = '0' then

```

```

327         rx_state      <= RX_RECV;      --Successful start condition recieved
328         rx_bit_idx    <= 0;
329         rx_bit_timer  <= RX_OVERSAMPLE - 1;
330     else
331         rx_state <= RX_IDLE;      --Start contidtion not satisfied
332     end if;      --remain in IDLE
333 else
334     rx_bit_timer <= rx_bit_timer - 1;
335 end if;
336 end if;
337
338 when RX_RECV =>
339     if rx_tick = '1' then
340         if rx_bit_timer = 0 then
341             rx_shift(rx_bit_idx) <= rx_sync;      --Read and place that bit
342
343             if rx_bit_idx = DATA_BITS - 1 then --If we read the last bit...
344                 if PARITY = 0 then      --Again no parity first
345                     rx_state      <= RX_STOP;
346                     rx_stop_cnt  <= STOP_BITS;
347                     rx_bit_timer <= RX_OVERSAMPLE - 1;
348                 else
349                     rx_state      <= RX_PAR;      --Otherwise on to read parity bit
350                     rx_bit_timer <= RX_OVERSAMPLE - 1;
351                 end if;
352             else
353                 rx_bit_idx    <= rx_bit_idx + 1;      --Increment bit index
354                 rx_bit_timer <= RX_OVERSAMPLE - 1;
355             end if;
356         else
357             rx_bit_timer <= rx_bit_timer - 1;
358         end if;
359     end if;
360
361 when RX_PAR =>
362     if rx_tick = '1' then
363         if rx_bit_timer = 0 then
364             rx_par_bit <= rx_sync;      --read in parity bit
365
366             pcalc := parity_xor(rx_shift, DATA_BITS); --calculate parity recieved
367             if PARITY = 1 then
368                 if rx_sync /= pcalc then
369                     rx_parity_err_r <= '1';      --Assert Parity Fail
370                 end if;
371             else
372                 if rx_sync /= (not pcalc) then
373                     rx_parity_err_r <= '1';      --Assert Parity Fail
374                 end if;
375             end if;
376
377             rx_state      <= RX_STOP;      --Check for the Stop Bit
378             rx_stop_cnt  <= STOP_BITS;
379             rx_bit_timer <= RX_OVERSAMPLE - 1;
380         else
381             rx_bit_timer <= rx_bit_timer - 1;
382         end if;
383     end if;
384
385 when RX_STOP =>      --Send that stop bit

```

```

386         if rx_tick = '1' then
387             if rx_bit_timer = 0 then
388                 if rx_sync /= '1' then
389                     rx_framing_err_r <= '1';
390                 end if;
391
392                 if rx_stop_cnt <= 1 then
393                     if rx_valid_r = '0' then
394                         rx_valid_r <= '1';
395                     else
396                         rx_overrun_r <= '1';
397                     end if;
398
399                     rx_state <= RX_IDLE;          --Send us back home...
400                 else
401                     rx_stop_cnt <= rx_stop_cnt - 1;
402                     rx_bit_timer <= RX_OVERSAMPLE - 1;
403                 end if;
404             else
405                 rx_bit_timer <= rx_bit_timer - 1;
406             end if;
407         end if;
408     end case;
409 end if;
410 end if;
411 end process;
412
413 end architecture;

```

A.12 My_Own_FN_FIFO.vhd

```

1  -- =====
2  -- File: My_Own_FN_FIFO.vhd
3  --
4  -- Creator: Kevin Lyon
5  -- Date Created: 05 March 2026
6  --
7  -- Description:
8  --   "My Own F'n FIFO" - simple parameterized synchronous FIFO.
9  --
10 --   Created after vendor FIFO showed unexpected Empty flag behavior that
11 --   caused state-machine timing issues. This implementation provides fully
12 --   predictable flag behavior.
13 --
14 -- Implementation Notes:
15 --   Single clock domain FIFO
16 --   push/pop are 1-clock pulses
17 --   dout is show-ahead (always displays next unread entry)
18 --   empty = count = 0
19 --   full  = count = DEPTH
20 --   DEPTH must be a power of two
21 --
22 -- Typical Use:
23 --   push <= data_valid
24 --   pop  <= consumer_ready
25 --
26 -- Revision History:

```

```

27 -- 2026-03-05 Initial version
28 -- =====
29
30 library ieee;
31 use ieee.std_logic_1164.all;
32 use ieee.numeric_std.all;
33
34 entity My_Own_FN_FIFO is
35     generic (
36         WIDTH : integer := 8;
37         DEPTH : integer := 16 -- MUST be power of 2
38     );
39     port (
40         clk      : in  std_logic;
41         rst      : in  std_logic; -- active-high
42
43         push     : in  std_logic; -- 1-clock pulse
44         din      : in  std_logic_vector(WIDTH-1 downto 0);
45
46         pop      : in  std_logic; -- 1-clock pulse
47         dout     : out std_logic_vector(WIDTH-1 downto 0);
48
49         empty    : out std_logic;
50         full     : out std_logic
51     );
52 end entity;
53
54 architecture rtl of My_Own_FN_FIFO is
55
56     function clog2(n : integer) return integer is
57         variable r : integer := 0;
58         variable v : integer := n - 1;
59     begin
60         while v > 0 loop
61             v := v / 2;
62             r := r + 1;
63         end loop;
64         return r;
65     end function;
66
67     constant ADDR_W : integer := clog2(DEPTH); --Just enough bits to count to DEPTH-1
68
69     type ram_t is array (0 to DEPTH-1) of std_logic_vector(WIDTH-1 downto 0);
70     signal ram : ram_t := (others => (others => '0'));
71
72     signal wptr : unsigned(ADDR_W-1 downto 0) := (others => '0');
73     signal rptr : unsigned(ADDR_W-1 downto 0) := (others => '0');
74     signal count : unsigned(ADDR_W downto 0) := (others => '0'); -- 0..DEPTH
75
76     constant COUNT0 : unsigned(count'length-1 downto 0) := (others => '0');
77     constant COUNT_MAX : unsigned(count'length-1 downto 0) := to_unsigned(DEPTH, count'length)
78         ;
79
80     -- internal flag signals (so we can read them)
81     signal empty_i : std_logic;
82     signal full_i : std_logic;
83
84     signal do_push : std_logic;
85     signal do_pop : std_logic;

```

```

85
86 begin
87
88     -- show-ahead output (dout always shows next unread entry)
89     dout <= ram(to_integer(rptr));
90
91     empty_i <= '1' when count = COUNT0    else '0';
92     full_i  <= '1' when count = COUNT_MAX else '0';
93
94     empty <= empty_i;
95     full  <= full_i;
96
97     -- only allow legal operations
98     do_push <= push and (not full_i);
99     do_pop  <= pop  and (not empty_i);
100
101     process(clk)
102     begin
103         if rising_edge(clk) then
104             if rst = '1' then
105                 wptr <= (others => '0');
106                 rptr <= (others => '0');
107                 count <= (others => '0');
108             else
109                 -- WRITE
110                 if do_push = '1' then
111                     ram(to_integer(wptr)) <= din;
112                     wptr <= wptr + 1;
113                 end if;
114
115                 -- READ POINTER ADVANCE
116                 if do_pop = '1' then
117                     rptr <= rptr + 1;
118                 end if;
119
120                 -- COUNT UPDATE
121                 if (do_push = '1') and (do_pop = '0') then
122                     count <= count + 1;
123                 elsif (do_push = '0') and (do_pop = '1') then
124                     count <= count - 1;
125                 else
126                     null; -- "00" or "11" => count unchanged
127                 end if;
128
129             end if;
130         end if;
131     end process;
132
133 end architecture;

```

A.13 AD7928_Core.vhd

```

1  -- =====
2  -- File: AD7928_Core.vhd
3  --
4  -- Creator: Kevin Lyon / ChatGPT
5  -- Date Created: 24 March 2026

```

```

6  -- Last Updated: 19 April 2026
7  --
8  -- Description:
9  --   SPI master for the AD7928 12-bit, 8-channel ADC.
10 --
11 --   This module generates one framed AD7928 SPI transaction per start pulse,
12 --   transmits the next control word on DIN, and captures the returned channel
13 --   tag and 12-bit conversion result from DOUT. The first completed frame is
14 --   treated as the expected pipeline-prime frame and does not assert
15 --   data_valid_o.
16 --
17 -- Notes:
18 --   - SCLK idles high so the first active clock edge is falling.
19 --   - The control word is built in addressed-channel mode with SEQ=0 and
20 --     SHADOW=0.
21 --   - The current Homework #9 configuration now matches the class slide's
22 --     range selection by using RANGE=0 for a 0V to 2*REFIN input span when
23 --     requested by the caller.
24 --
25 -- Revision History:
26 --
27 --   2026-03-24
28 --     - Initial SPI master implementation for the AD7928 interface.
29 --
30 --   2026-04-19
31 --     - The surrounding ADC client now drives RANGE=0 to match the class
32 --       example.
33 -- =====
34 library ieee;
35 use ieee.std_logic_1164.all;
36 use ieee.numeric_std.all;
37
38 entity ad7928_spi_master is
39   generic (
40     -- Number of system clock cycles for each HALF SCLK period.
41     -- Example:
42     -- clk = 25 MHz, CLK_DIV = 5  => SCLK = 5 MHz
43     -- Keep SCLK <= 20 MHz per datasheet.
44     CLK_DIV : positive := 5
45   );
46   port (
47     -----
48     -- System
49     -----
50     clk          : in  std_logic;
51     reset_i      : in  std_logic;
52
53     -----
54     -- User control
55     -----
56     start_i      : in  std_logic;          -- pulse to begin one conversion
57     frame
58     channel_i    : in  std_logic_vector(2 downto 0);  -- next channel to request
59     range_i      : in  std_logic := '0';          -- '1' = 0..REFIN, '0' = 0..2*REFIN
60     coding_i     : in  std_logic := '1';          -- '1' = straight binary, '0' =
61     twos complement
62
63     -----
64     -- Status / outputs

```

```

63  -----
64  busy_o          : out std_logic;
65  data_valid_o    : out std_logic;          -- pulses high when result_o is
        updated
66  primed_o        : out std_logic;          -- goes high after first dummy
        frame
67
68  result_o         : out std_logic_vector(11 downto 0); -- 12-bit ADC result
69  result_chan_o    : out std_logic_vector(2 downto 0); -- channel tag returned by ADC
70  raw_word_o       : out std_logic_vector(15 downto 0); -- full received 16-bit frame
71
72  -----
73  -- AD7928 SPI pins
74  -----
75  adc_cs_n_o       : out std_logic;
76  adc_sclk_o       : out std_logic;
77  adc_din_o        : out std_logic;
78  adc_dout_i       : in  std_logic
79  );
80  end entity;
81
82  architecture rtl of ad7928_spi_master is
83
84  type state_t is (
85      IDLE,
86      CS_SETUP,      -- CS low, SCLK high, capture leading zero before first falling edge
87      FALL_EDGE,     -- generate falling edge
88      LOW_PHASE,     -- hold SCLK low
89      RISE_EDGE,     -- generate rising edge
90      HIGH_PHASE,    -- hold SCLK high, sample DOUT before next falling edge
91      FINISH         -- end of frame / latch results
92  );
93
94  signal state : state_t := IDLE;
95
96  signal cs_n_reg      : std_logic := '1';
97  signal sclk_reg      : std_logic := '1'; -- idle high so first active edge is falling
98  signal din_reg       : std_logic := '0';
99
100 signal busy_reg       : std_logic := '0';
101 signal data_valid_reg : std_logic := '0';
102 signal primed_reg     : std_logic := '0';
103
104 signal result_reg      : std_logic_vector(11 downto 0) := (others => '0');
105 signal result_chan_reg : std_logic_vector(2 downto 0) := (others => '0');
106 signal raw_word_reg    : std_logic_vector(15 downto 0) := (others => '0');
107
108 signal tx_word         : std_logic_vector(15 downto 0) := (others => '0');
109 signal rx_word         : std_logic_vector(15 downto 0) := (others => '0');
110
111 signal half_cnt        : natural range 0 to CLK_DIV-1 := 0;
112 signal fall_count      : natural range 0 to 16 := 0; -- number of falling SCLK edges already
        generated
113
114  -----
115  -- Build the 16-bit DIN frame.
116  -- Bits [15:4] are the 12-bit control register:
117  --   11 WRITE   = 1
118  --   10 SEQ     = 0

```

```

119 -- 9 DC      = 0
120 -- 8:6 ADD2:0 = channel_i
121 -- 5:4 PM1:0  = 11 (normal operation)
122 -- 3 SHADOW   = 0
123 -- 2 DC      = 0
124 -- 1 RANGE    = range_i
125 -- 0 CODING   = coding_i
126 -- Bits [3:0] are non-implemented / zeros.
127 -----
128 function make_ctrl_word(
129     ch      : std_logic_vector(2 downto 0);
130     rangeb  : std_logic;
131     codeb   : std_logic
132 ) return std_logic_vector is
133     variable w : std_logic_vector(15 downto 0);
134 begin
135     w := (others => '0');
136
137     -- 12-bit control word in bits 15 downto 4
138     w(15)      := '1';      -- WRITE
139     w(14)      := '0';      -- SEQ = 0
140     w(13)      := '0';      -- don't care
141     w(12 downto 10) := ch;   -- ADD2..ADD0
142     w(9)       := '1';      -- PM1 = 1
143     w(8)       := '1';      -- PM0 = 1 => normal operation
144     w(7)       := '0';      -- SHADOW = 0
145     w(6)       := '0';      -- don't care
146     w(5)       := rangeb;   -- RANGE
147     w(4)       := codeb;    -- CODING
148
149     -- w(3 downto 0) left as zero
150     return w;
151 end function;
152
153 begin
154
155     busy_o      <= busy_reg;
156     data_valid_o <= data_valid_reg;
157     primed_o    <= primed_reg;
158
159     result_o     <= result_reg;
160     result_chan_o <= result_chan_reg;
161     raw_word_o   <= raw_word_reg;
162
163     adc_cs_n_o   <= cs_n_reg;
164     adc_sclk_o   <= sclk_reg;
165     adc_din_o    <= din_reg;
166
167     process(clk)
168     begin
169         if rising_edge(clk) then
170
171             if reset_i = '1' then
172                 state      <= IDLE;
173                 cs_n_reg    <= '1';
174                 sclk_reg    <= '1';
175                 din_reg     <= '0';
176                 busy_reg    <= '0';
177                 data_valid_reg <= '0';

```

```

178     primed_reg      <= '0';
179     result_reg       <= (others => '0');
180     result_chan_reg  <= (others => '0');
181     raw_word_reg     <= (others => '0');
182     rx_word          <= (others => '0');
183     tx_word          <= (others => '0');
184     half_cnt         <= 0;
185     fall_count       <= 0;
186
187 else
188     -- default pulse behavior
189     data_valid_reg <= '0';
190
191 case state is
192
193     -----
194     when IDLE =>
195         cs_n_reg  <= '1';
196         sclk_reg  <= '1';
197         din_reg   <= '0';
198         busy_reg  <= '0';
199         half_cnt  <= 0;
200         fall_count <= 0;
201
202         if start_i = '1' then
203             busy_reg  <= '1';
204             cs_n_reg  <= '0'; -- CS falling edge starts conversion
205             sclk_reg  <= '1'; -- start with clock high
206             tx_word   <= make_ctrl_word(channel_i, range_i, coding_i);
207             din_reg   <= tx_word(15); -- WRITE bit before first falling edge
208             rx_word   <= (others => '0');
209             half_cnt  <= 0;
210             fall_count <= 0;
211             state     <= CS_SETUP;
212         end if;
213
214     -----
215     -- CS is already low. Allow one half-period with SCLK high.
216     -- During this phase, the ADC presents the leading zero on DOUT.
217     -----
218     when CS_SETUP =>
219         if half_cnt = CLK_DIV-1 then
220             half_cnt <= 0;
221             rx_word(15) <= adc_dout_i; -- leading zero is available after CS goes low
222             state <= FALL_EDGE;
223         else
224             half_cnt <= half_cnt + 1;
225         end if;
226
227     -----
228     -- Generate one falling edge. ADC clocks out next bit on DOUT and
229     -- clocks in current DIN bit on this falling edge.
230     -----
231     when FALL_EDGE =>
232         sclk_reg <= '0';
233         if fall_count < 16 then
234             fall_count <= fall_count + 1;
235         end if;
236         half_cnt <= 0;

```

```

237         state      <= LOW_PHASE;
238
239 -----
240 when LOW_PHASE =>
241     if half_cnt = CLK_DIV-1 then
242         half_cnt <= 0;
243         state    <= RISE_EDGE;
244     else
245         half_cnt <= half_cnt + 1;
246     end if;
247
248 -----
249 when RISE_EDGE =>
250     sclk_reg <= '1';
251
252     -- Present next DIN bit while clock is high, before next falling edge
253     -- Only meaningful through the first 16 transmitted bits.
254     if fall_count <= 15 then
255         din_reg <= tx_word(15 - fall_count);
256     else
257         din_reg <= '0';
258     end if;
259
260     half_cnt <= 0;
261
262     if fall_count = 16 then
263         state <= FINISH;
264     else
265         state <= HIGH_PHASE;
266     end if;
267
268 -----
269 -- During high phase, sample the DOUT bit that was launched by the
270 -- previous falling edge.
271 --
272 -- Mapping:
273 --   rx_word(15) captured in CS_SETUP  = leading zero
274 --   after 1st falling edge, capture rx_word(14) = ADD2
275 --   ...
276 --   after 15th falling edge, capture rx_word(0)  = DB0
277 --   16th falling edge ends the transfer / DOUT tri-states
278 -----
279 when HIGH_PHASE =>
280     if half_cnt = CLK_DIV-1 then
281         half_cnt <= 0;
282
283         if (fall_count >= 1) and (fall_count <= 15) then
284             rx_word(15 - fall_count) <= adc_dout_i;
285         end if;
286
287         state <= FALL_EDGE;
288     else
289         half_cnt <= half_cnt + 1;
290     end if;
291
292 -----
293 when FINISH =>
294     cs_n_reg <= '1';
295     sclk_reg <= '1';

```

```

296     busy_reg    <= '0';
297
298     raw_word_reg <= rx_word;
299
300     -- AD7928 frame format:
301     --   [15]    leading zero
302     --   [14:12] channel ID
303     --   [11:0]  conversion result
304     --
305     -- Result read back corresponds to the PREVIOUS configuration word.
306     result_chan_reg <= rx_word(14 downto 12);
307     result_reg      <= rx_word(11 downto 0);
308
309     -- First completed frame is just the pipeline-prime/dummy result.
310     if primed_reg = '1' then
311         data_valid_reg <= '1';
312     else
313         primed_reg      <= '1';
314         data_valid_reg <= '0';
315     end if;
316
317     state <= IDLE;
318
319     end case;
320 end if;
321 end if;
322 end process;
323
324 end architecture;

```